



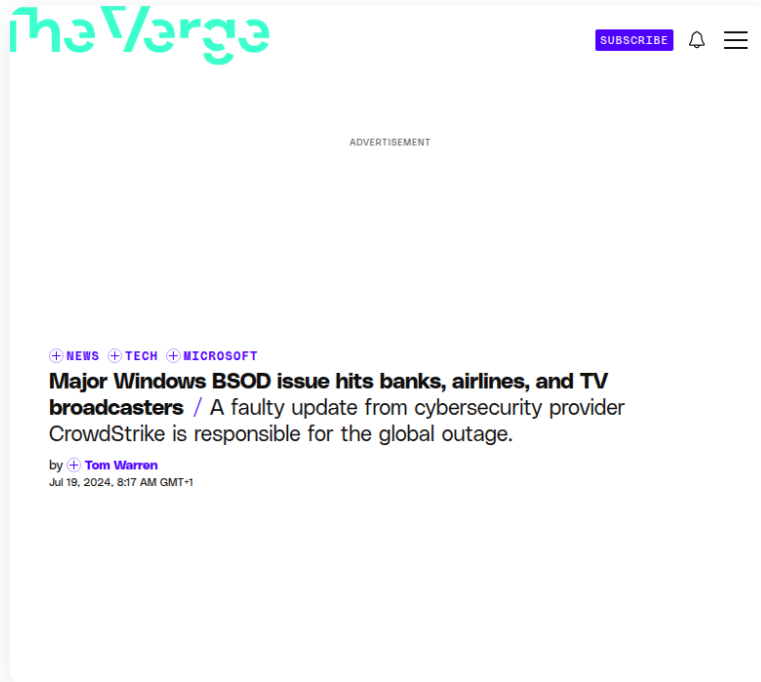
Rethinking the Evaluation of Microservice RCA with a Fault Propagation-Aware Benchmark

Aoyang Fang, Songhan Zhang, Yifan Yang, Haotong Wu, Junjielong Xu
Xuyang Wang, Rui Wang, Manyi Wang, Qisheng Lu, Pinjia He

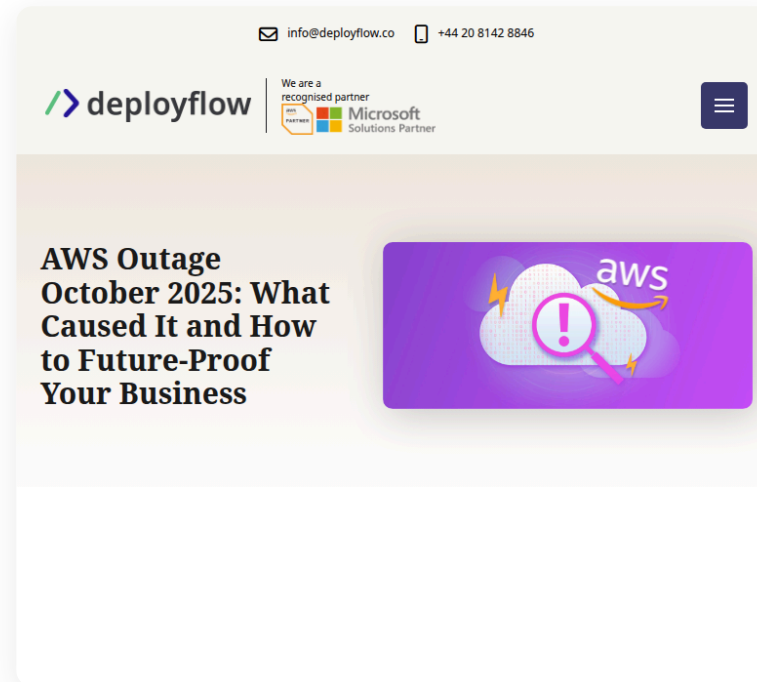
The Chinese University of Hong Kong, Shenzhen

School of Data Science | School of Artificial Intelligence

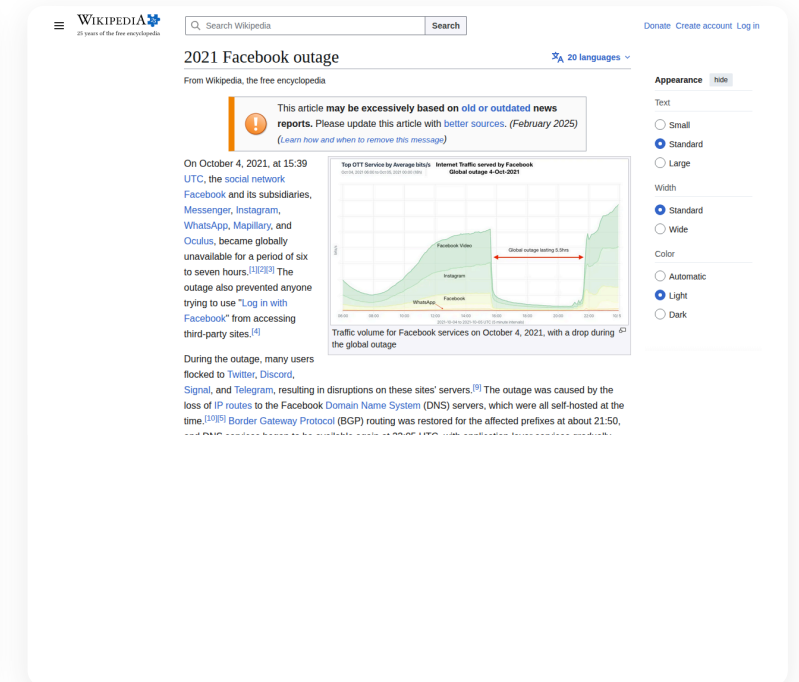
Cloud incidents cost billions



2024.07 · CrowdStrike
~\$5.4B Fortune 500 losses

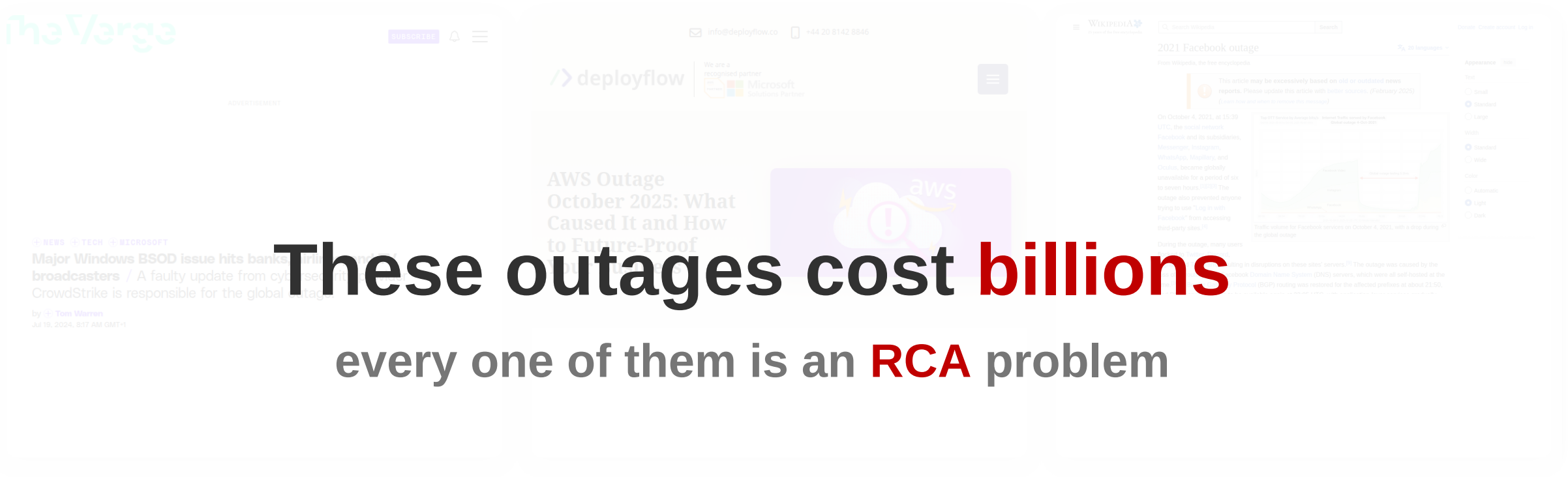


2025.10 · AWS US-EAST-1
>\$1.1B · 15h outage · 1000+ companies



2021.10 · Meta
~\$100M ad revenue · 6h · 3.5B users

Cloud incidents cost billions



These outages cost billions

every one of them is an RCA problem

2024.07 • CrowdStrike
~\$5.4B Fortune 500 losses

2025.10 • AWS US-EAST-1
>\$1.1B • 15h outage • 1000+ companies

2021.10 • Meta
~\$100M ad revenue • 6h • 3.5B users

AWS 2025: A DNS race cascaded through 6 layers

AWS 2025: A DNS race cascaded through 6 layers

ROOT CAUSE



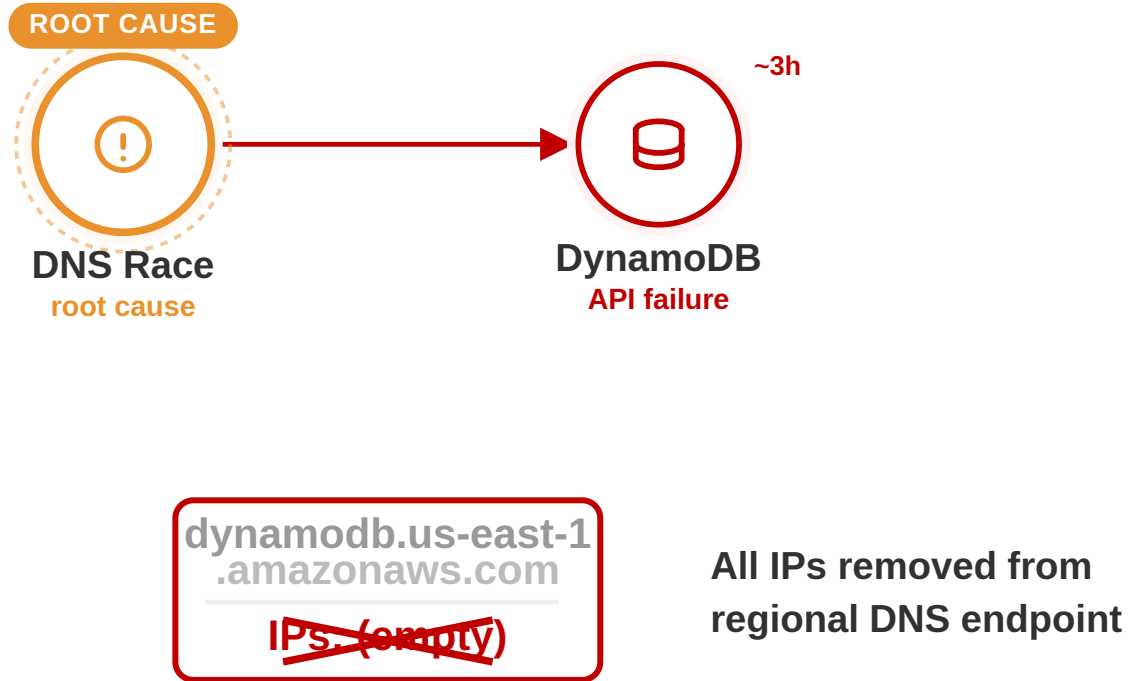
DNS Race
root cause



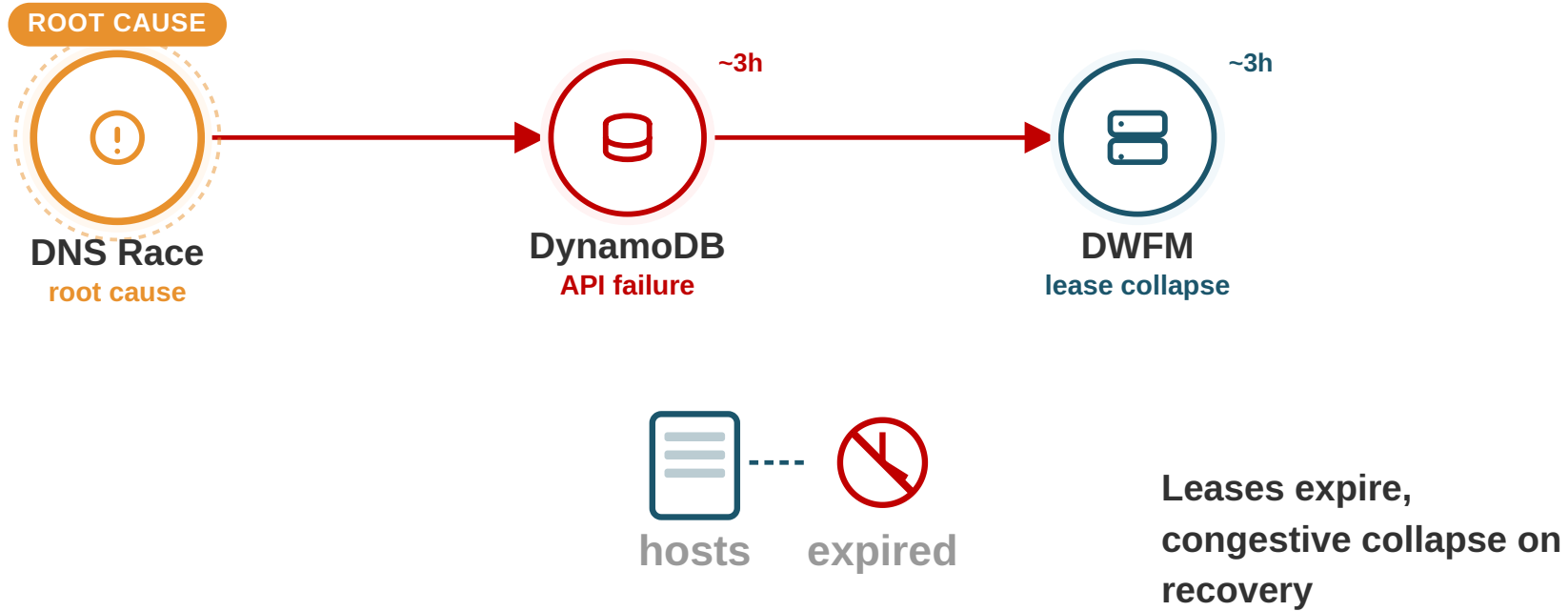
DNS Worker

Two DNS workers race on
the same endpoint

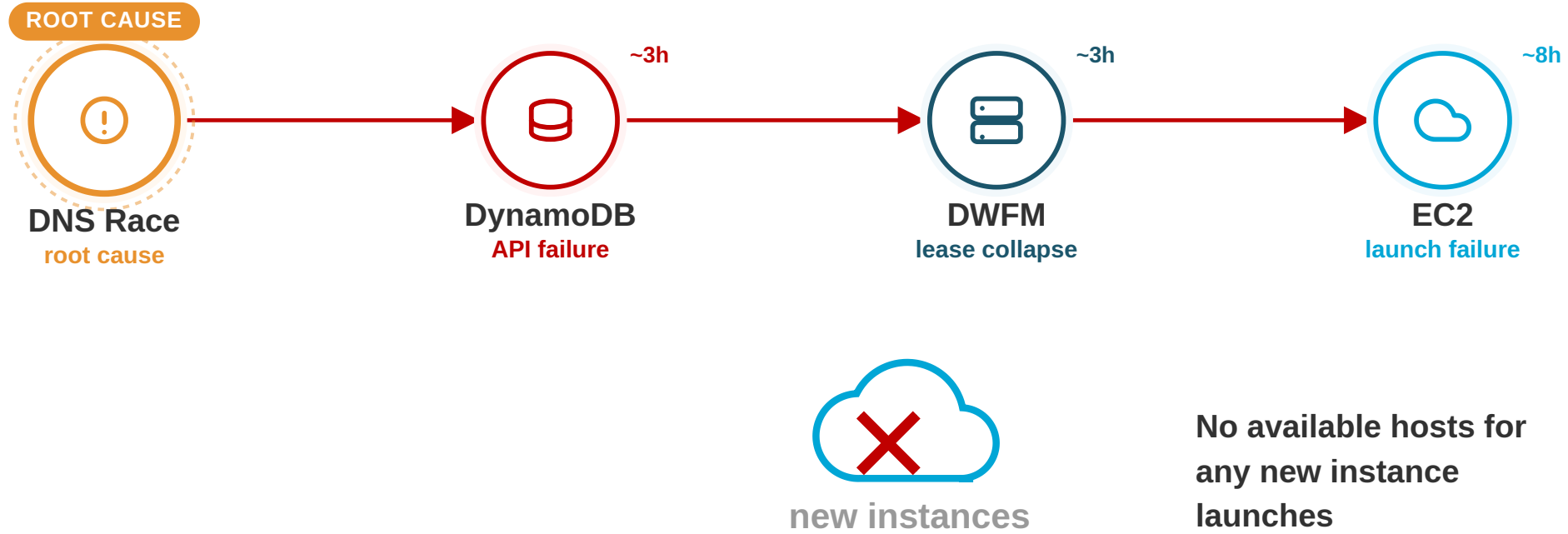
AWS 2025: A DNS race cascaded through 6 layers



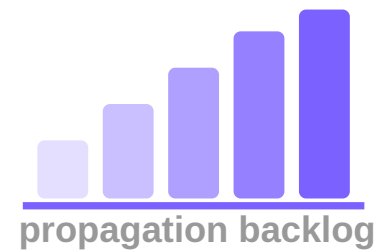
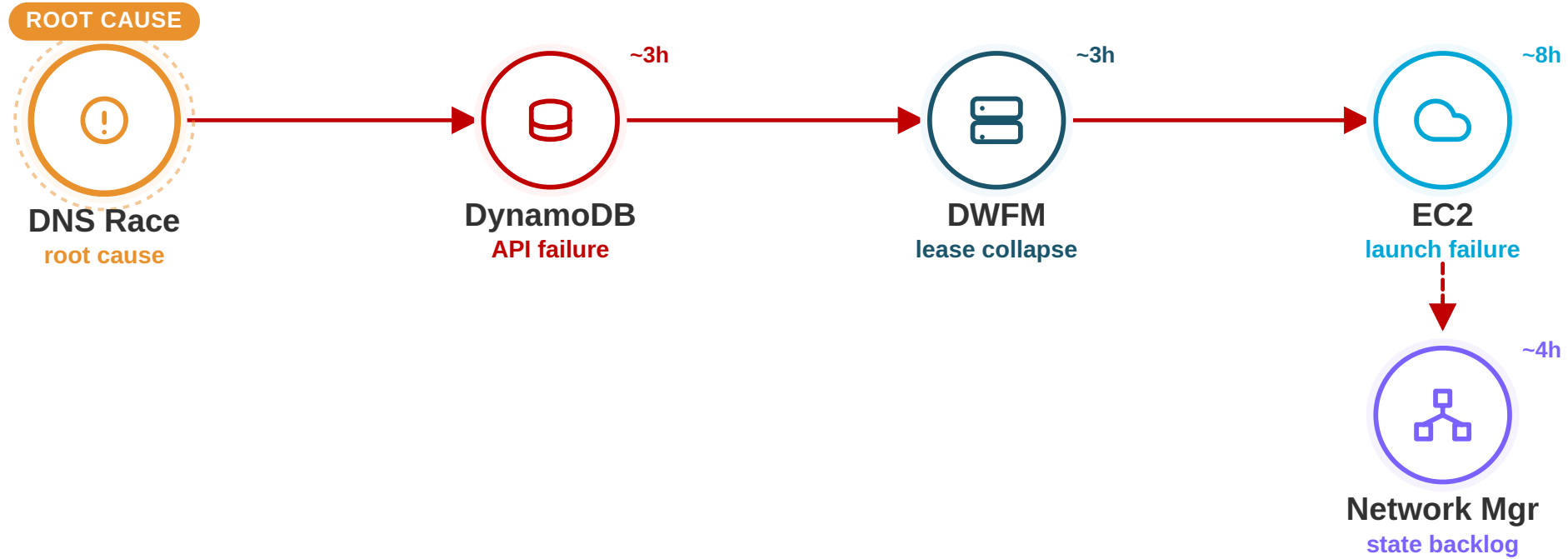
AWS 2025: A DNS race cascaded through 6 layers



AWS 2025: A DNS race cascaded through 6 layers

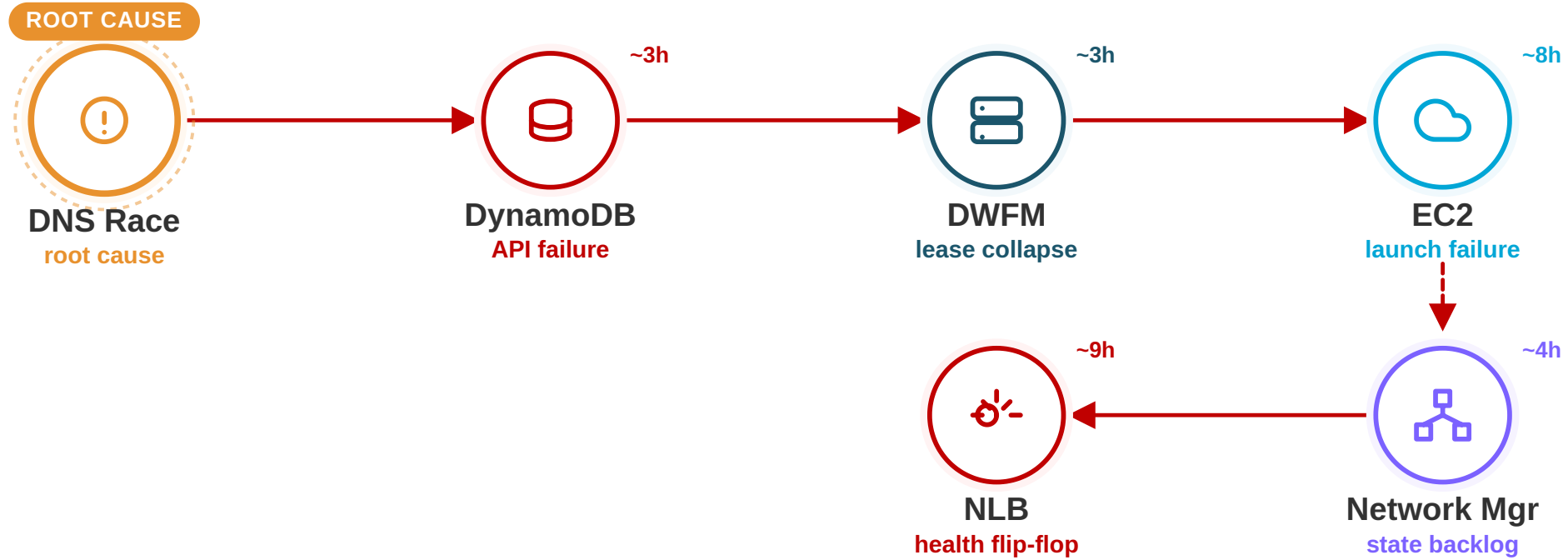


AWS 2025: A DNS race cascaded through 6 layers



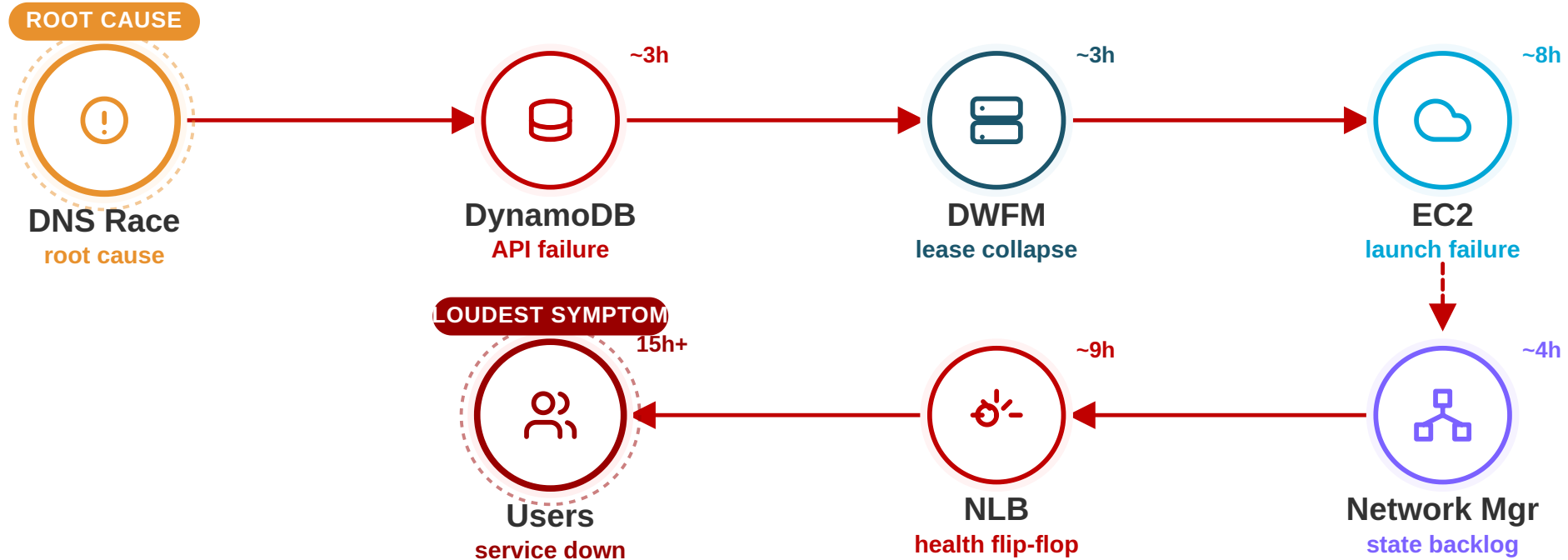
Network state changes
queue faster than
processed

AWS 2025: A DNS race cascaded through 6 layers



Health checks flip
between healthy and
unhealthy

AWS 2025: A DNS race cascaded through 6 layers



503 503 503
5h+ total outage

Snapchat, Zoom, Slack ...
connection timeouts,
service down

How RCA works in papers

How RCA works in papers



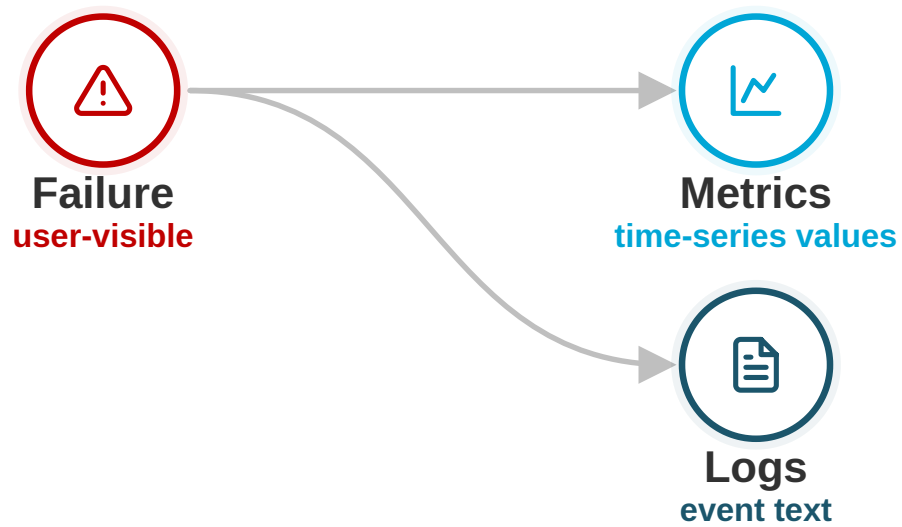
Failure
user-visible

How RCA works in papers



Numeric values over time:
CPU, latency, error rate ...

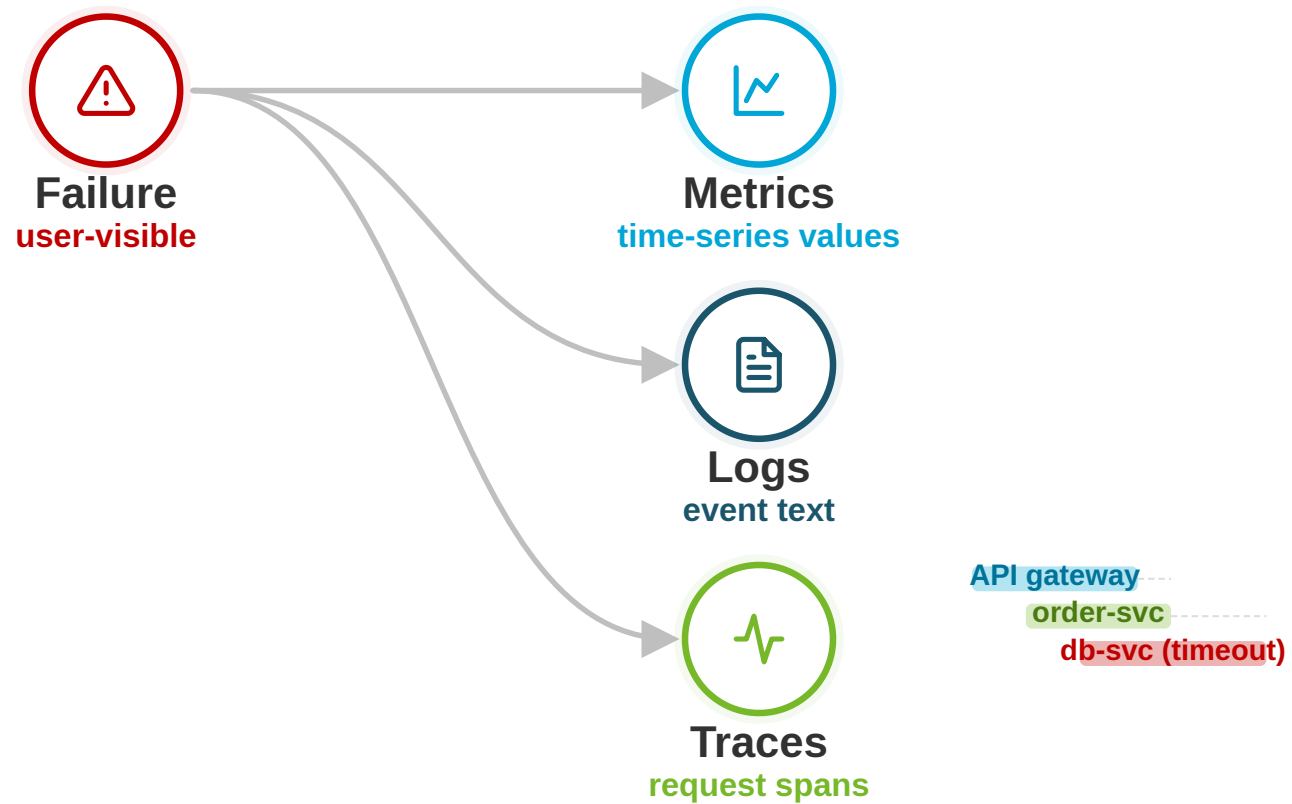
How RCA works in papers



```
10:03:21 INFO request completed 200
10:03:22 WARN latency exceeded 500ms
10:03:23 ERROR connection refused db-svc
10:03:24 INFO retry attempt 1/3
```

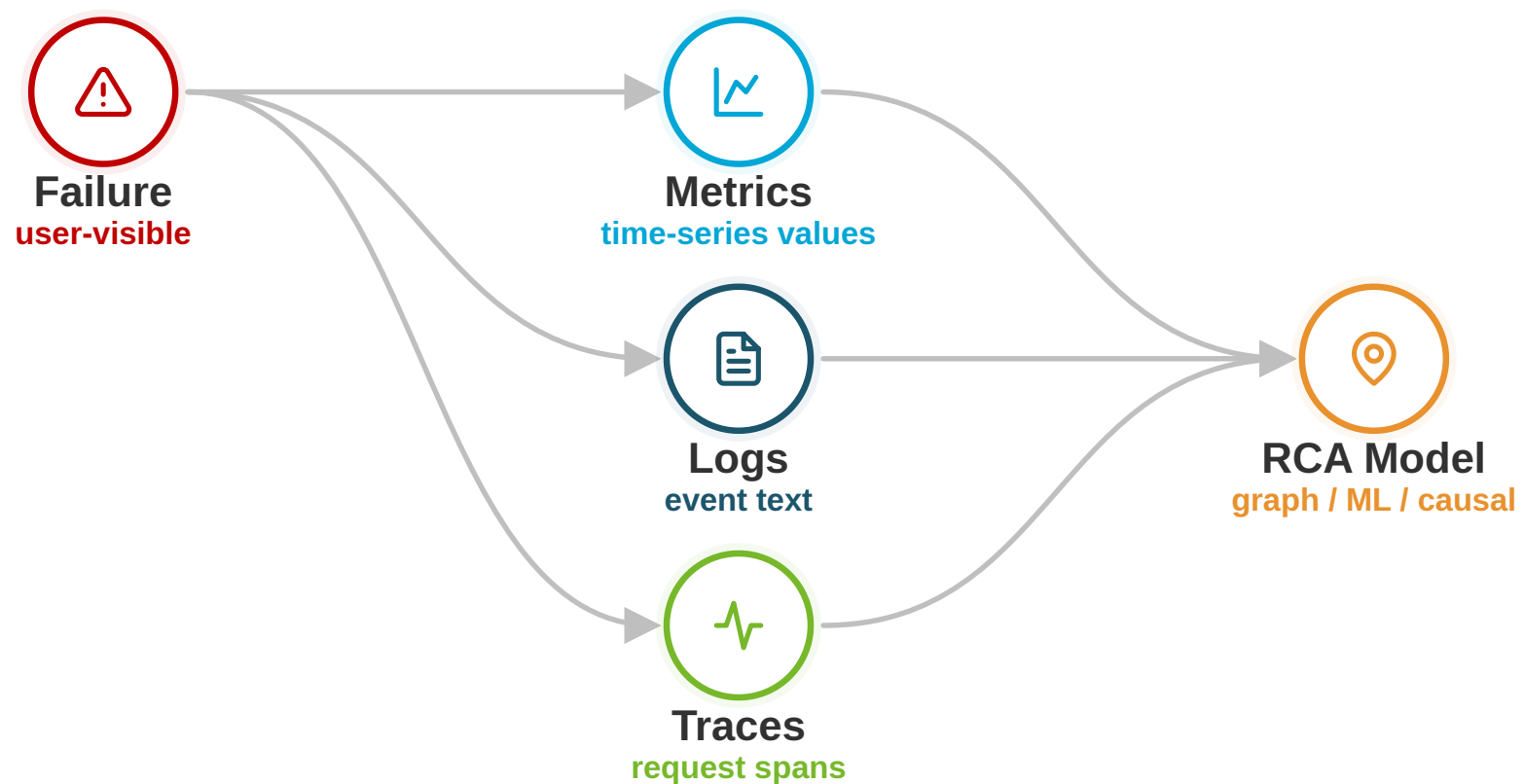
Structured event text:
errors, warnings,
exceptions ...

How RCA works in papers

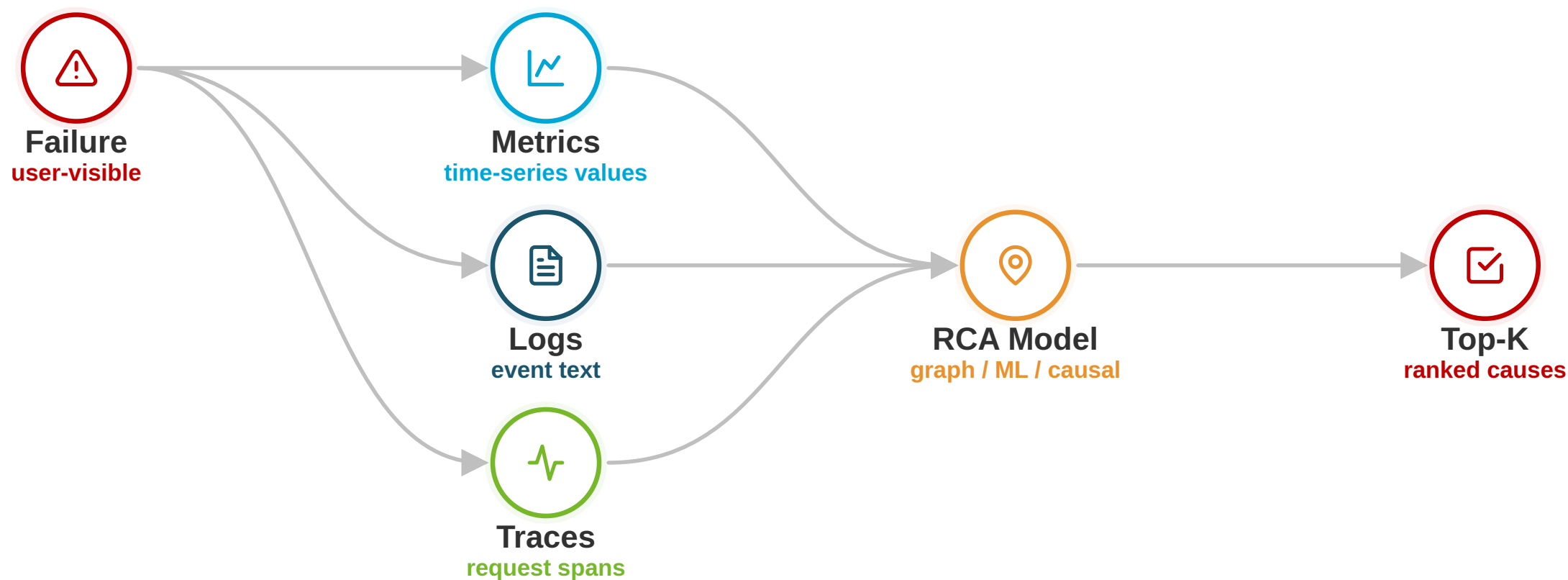


Request path across services: which span is slow or failing

How RCA works in papers



How RCA works in papers



Preliminary Study

Are existing benchmarks challenging enough?



RQ1 Are benchmarks realistic?

Preliminary Study

Are existing benchmarks challenging enough?



RQ1 Are benchmarks realistic?

RQ2 Simple baseline vs. SOTA?

Preliminary Study

Are existing benchmarks challenging enough?



RQ1 Are benchmarks realistic?

RQ2 Simple baseline vs. SOTA?

RQ3 Why does the shortcut work?

What makes real incidents hard?

What makes real incidents hard?



Fault propagation
cascades across service layers

What makes real incidents hard?



Fault propagation

cascades across service layers



Misleading symptoms

loudest signal $\neq$ root cause

What makes real incidents hard?



Fault propagation

cascades across service layers



Misleading symptoms

loudest signal $\neq$ root cause

Yet most existing benchmarks:

30-159

cases

2-3

max depth

1-2

fault types

99%

incomplete obs.

What makes real incidents hard?



Fault propagation

cascades across service layers



Misleading symptoms

loudest signal $\neq$ root cause

Yet most existing benchmarks:

30-159

cases

2-3

max depth

1-2

fault types

99%

incomplete obs.

Do existing benchmarks preserve these properties?

What makes real incidents hard?



Fault propagation

cascades across service layers



Misleading symptoms

loudest signal $\neq$ root cause

Yet most existing benchmarks:

30-159

cases

2-3

max depth

1-2

fault types

99%

incomplete obs.

Do existing benchmarks preserve these properties?

To find out, we designed a deliberately **simple probe**

SimpleRCA: a deliberately simple probe

SimpleRCA: a deliberately simple probe



**Metric values above
threshold counted as
alerts**

SimpleRCA: a deliberately simple probe



Metrics
threshold check



Traces
latency check

gateway 12ms

order-svc 8ms

db-svc 580ms !

Span latency above
threshold counted as
alerts

SimpleRCA: a deliberately simple probe



Metrics
threshold check



Traces
latency check



Logs
keyword match

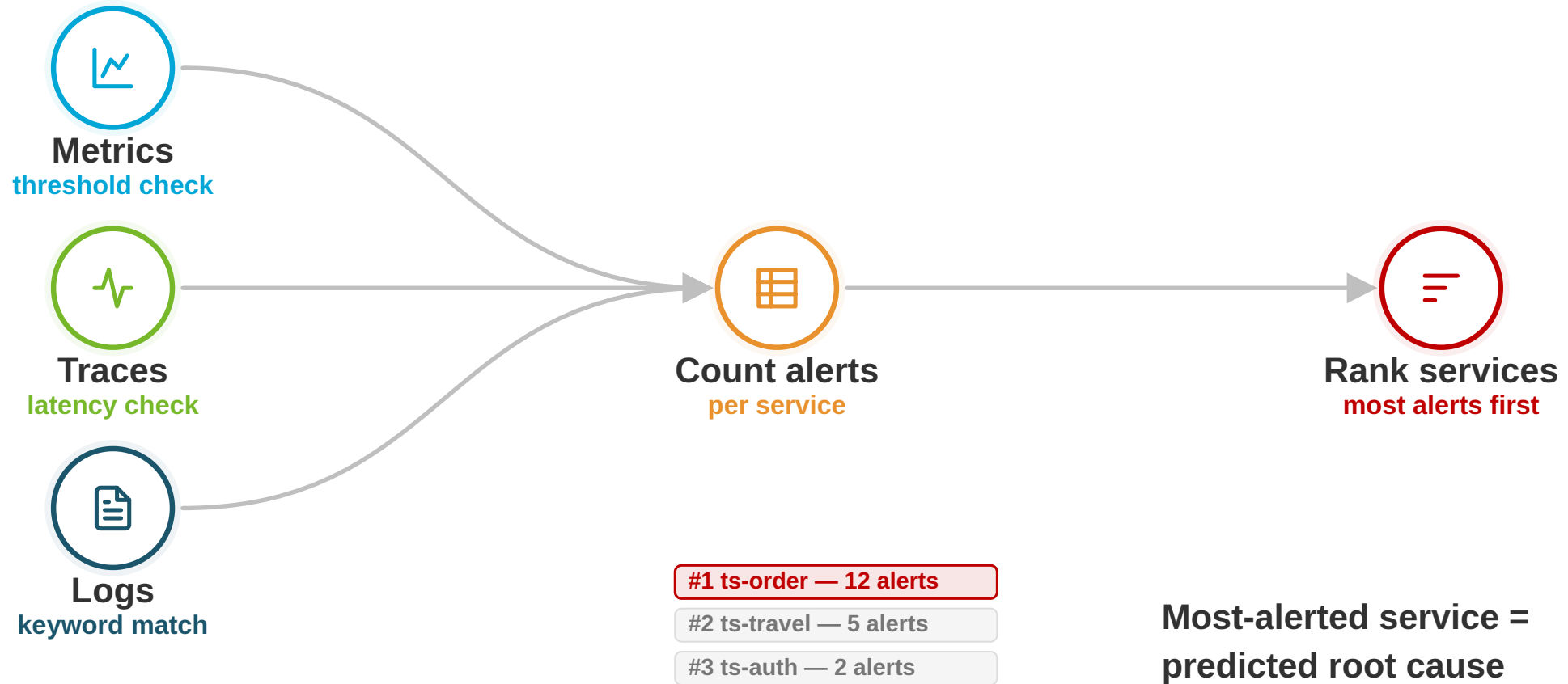
```
10:03:21 INFO request ok
10:03:22 ERROR conn refused
10:03:23 INFO retry 1/3
10:03:24 ERROR timeout
```

Lines matching error
keywords counted as
alerts

SimpleRCA: a deliberately simple probe



SimpleRCA: a deliberately simple probe



The shortcut performs too well

Dataset	SimpleRCA			Method	Best SOTA		
	@1	@3	@5		@1	@3	@5
RE2-OB	0.47	0.93	1.00	BARO	0.14	0.93	0.98
RE2-TT	0.80	0.87	0.93	BARO	0.67	0.84	0.89
RE2-SS	0.24	0.88	0.97	BARO	0.14	0.82	0.94
RE3-OB	0.30	0.90	0.90	BARO	0.00	0.90	0.90
RE3-TT	0.83	1.00	1.00	BARO	0.50	0.93	1.00
RE3-SS	0.83	0.90	0.90	BARO	0.00	0.83	0.93
Nezha-TT	0.93	0.98	0.98	Nezha	0.87	0.98	0.98
Nezha-OB	0.91	0.91	0.91	Nezha	0.93	0.96	0.96
Eadro-TT	0.81	0.83	0.83	Eadro *	0.99	0.99	0.99
Eadro-SN	0.83	0.83	0.83	Eadro *	0.97	0.99	0.99
AIOps-21	0.63	0.82	0.91	ART	0.72	0.89	—

* Eadro: training / test set overlap

The shortcut performs too well

Dataset	SimpleRCA			Method	Best SOTA		
	@1	@3	@5		@1	@3	@5
RE2-OB	0.47	0.93	1.00	BARO	0.14	0.93	0.98
RE2-TT	0.80	0.87	0.93	BARO	0.67	0.84	0.89
RE2-SS	0.24	0.88	0.97	BARO	0.14	0.82	0.94
RE3-OB	0.30	0.90	0.90	BARO	0.00	0.90	0.90
RE3-TT	0.83	1.00	1.00	BARO	0.50	0.93	1.00
RE3-SS	0.83	0.90	0.90	BARO	0.00	0.83	0.93
Nezha-TT	0.93	0.98	0.98	Nezha	0.87	0.98	0.98
Nezha-OB	0.91	0.91	0.91	Nezha	0.93	0.96	0.96
Eadro-TT	0.81	0.83	0.83	Eadro *	0.99	0.99	0.99
Eadro-SN	0.83	0.83	0.83	Eadro *	0.97	0.99	0.99
AIOps-21	0.63	0.82	0.91	ART	0.72	0.89	—

* Eadro: training / test set overlap

SimpleRCA wins matches or beats published SOTA on 7 datasets

The shortcut performs too well

Dataset	SimpleRCA			Method	Best SOTA		
	@1	@3	@5		@1	@3	@5
RE2-OB	0.47	0.93	1.00	BARO	0.14	0.93	0.98
RE2-TT	0.80	0.87	0.93	BARO	0.67	0.84	0.89
RE2-SS	0.24	0.88	0.97	BARO	0.14	0.82	0.94
RE3-OB	0.30	0.90	0.90	BARO	0.00	0.90	0.90
RE3-TT	0.83	1.00	1.00	BARO	0.50	0.93	1.00
RE3-SS	0.83	0.90	0.90	BARO	0.00	0.83	0.93
Nezha-TT	0.93	0.98	0.98	Nezha	0.87	0.98	0.98
Nezha-OB	0.91	0.91	0.91	Nezha	0.93	0.96	0.96
Eadro-TT	0.81	0.83	0.83	Eadro *	0.99	0.99	0.99
Eadro-SN	0.83	0.83	0.83	Eadro *	0.97	0.99	0.99
AIOps-21	0.63	0.82	0.91	ART	0.72	0.89	—

* Eadro: training / test set overlap

Even when losing $\text{gap} \leq 0.18$ · Eadro* has data leakage

The shortcut performs too well

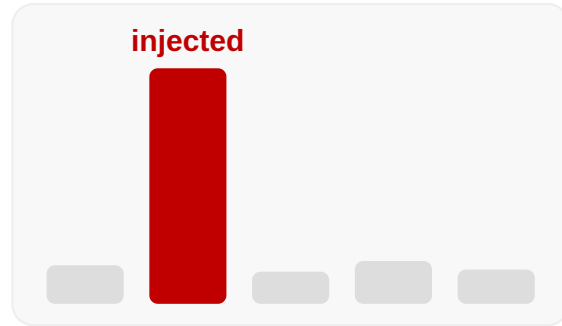
Dataset	SimpleRCA			Method	Best SOTA		
	@1	@3	@5		@1	@3	@5
RE2-OB	0.47	0.93	1.00	BARO	0.14	0.93	0.98
RE2-TT	0.80	0.87	0.93	BARO	0.67	0.84	0.89
RE2-SS	0.24	0.88	0.97	BARO	0.14	0.82	0.94
RE3-OB	0.30	0.90	0.90	BARO	0.00	0.90	0.90
RE3-TT	0.83	1.00	1.00	BARO	0.50	0.93	1.00
RE3-SS	0.83	0.90	0.90	BARO	0.00	0.83	0.93
Nezha-TT	0.93	0.98	0.98	Nezha	0.87	0.98	0.98
Nezha-OB	0.91	0.91	0.91	Nezha	0.93	0.96	0.96
Eadro-TT	0.81	0.83	0.83	Eadro *	0.99	0.99	0.99
Eadro-SN	0.83	0.83	0.83	Eadro *	0.97	0.99	0.99
AIOps-21	0.63	0.82	0.91	ART	0.72	0.89	—

* Eadro: training / test set overlap

7 / 11 A trivial probe should not match published models

Why does the shortcut work?

Why does the shortcut work?



68%

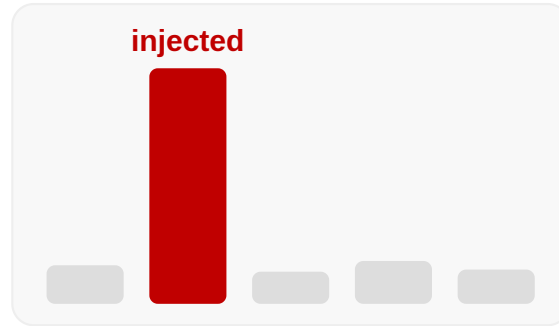
Type I

Localized

Only injected service shows anomaly

trivially easy to find

Why does the shortcut work?



68%

Type I

Localized

Only injected service shows anomaly
trivially easy to find



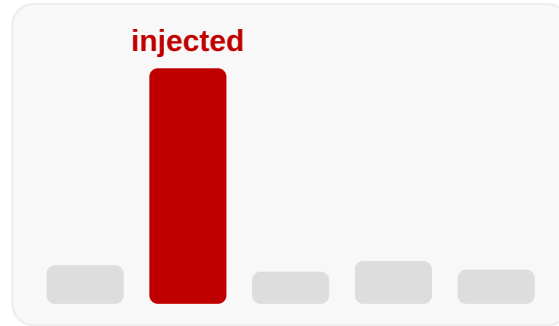
18%

Type II

Under-expressed

No service shows any signal
impossible for any method

Why does the shortcut work?



68%

Type I

Localized

Only injected service shows anomaly
trivially easy to find

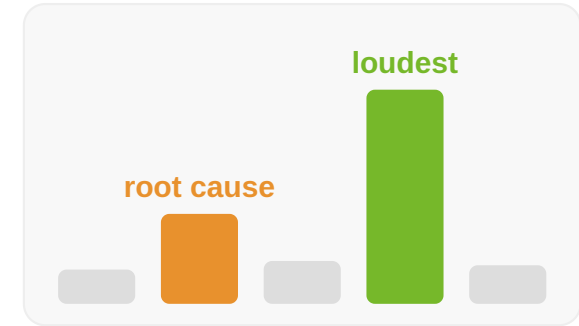


18%

Type II

Under-expressed

No service shows any signal
impossible for any method



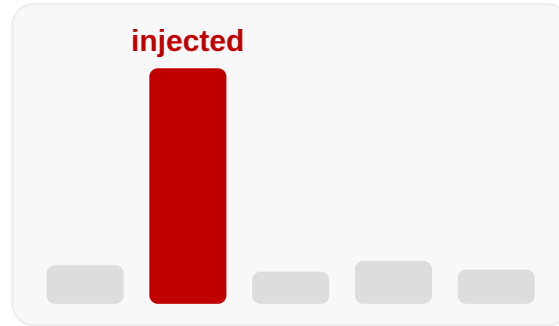
14%

Type III

Propagated

Downstream services are louder
requires causal reasoning

Why does the shortcut work?



68%

Type I

Localized

Only injected service shows anomaly
trivially easy to find

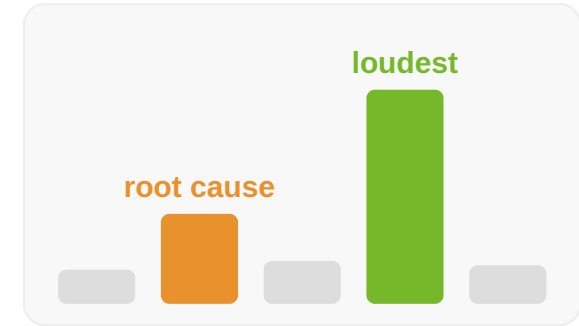


18%

Type II

Under-expressed

No service shows any signal
impossible for any method



14%

Type III

Propagated

Downstream services are louder
requires causal reasoning

68% trivially easy + **18%** no signal = **86%** don't test causal reasoning

99% of cases lack fault-relevant observability data

Benchmark Construction

A fault propagation-aware benchmark generator



01 Realistic fault scenarios

Benchmark Construction

A fault propagation-aware benchmark generator



01 Realistic fault scenarios

02 Impact-driven validation

Benchmark Construction

A fault propagation-aware benchmark generator



01 Realistic fault scenarios

02 Impact-driven validation

03 Automated & reproducible

What does a realistic benchmark need?

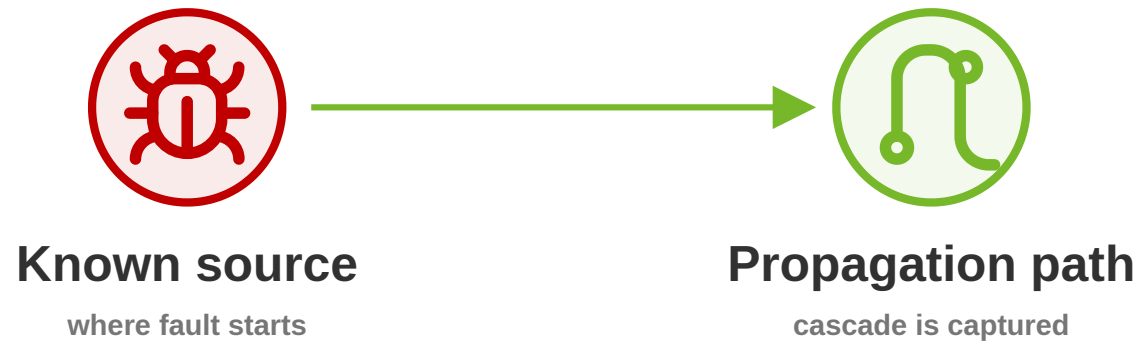
What does a realistic benchmark need?



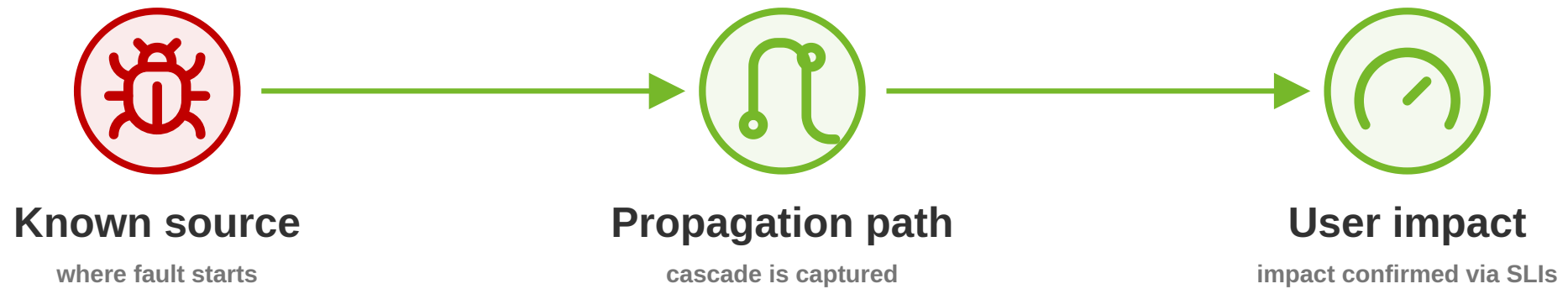
Known source

where fault starts

What does a realistic benchmark need?

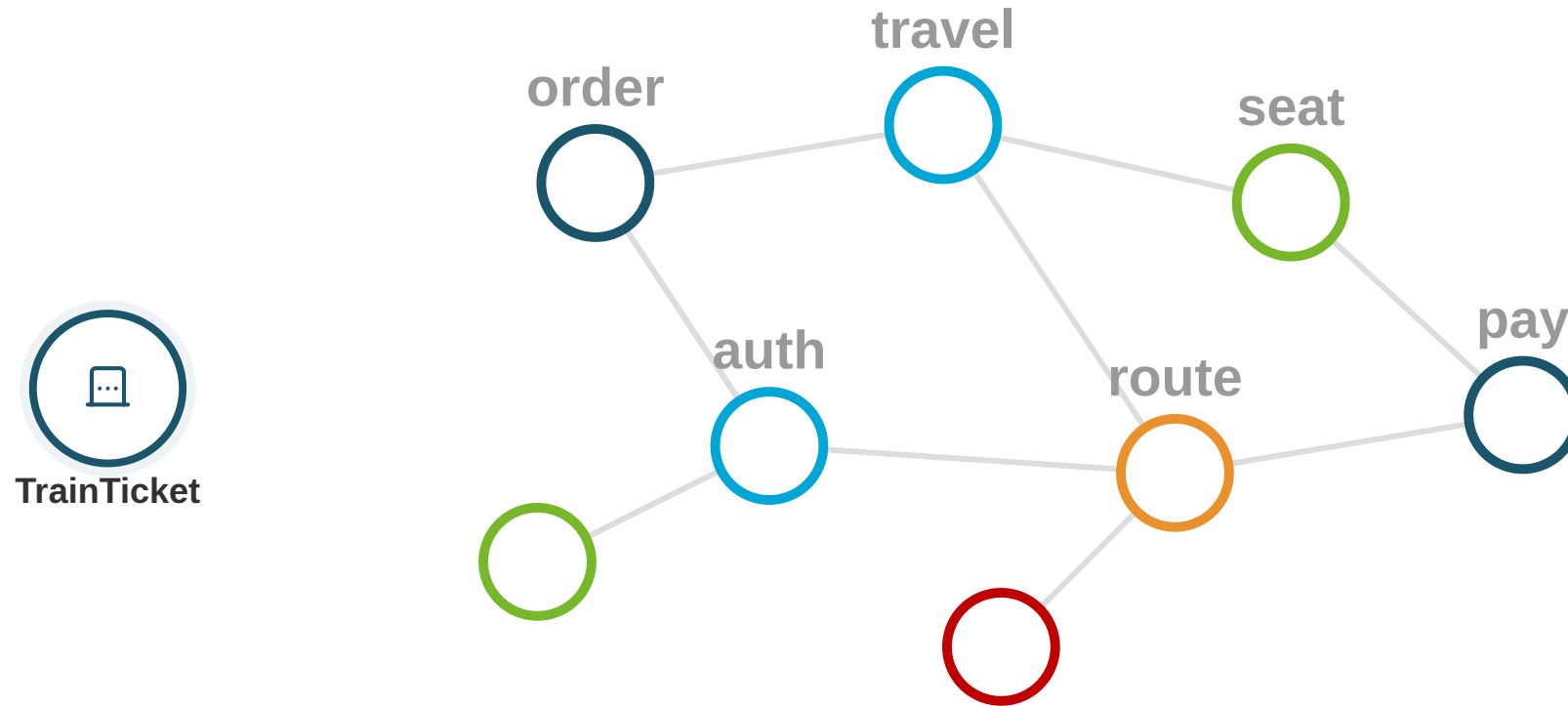


What does a realistic benchmark need?



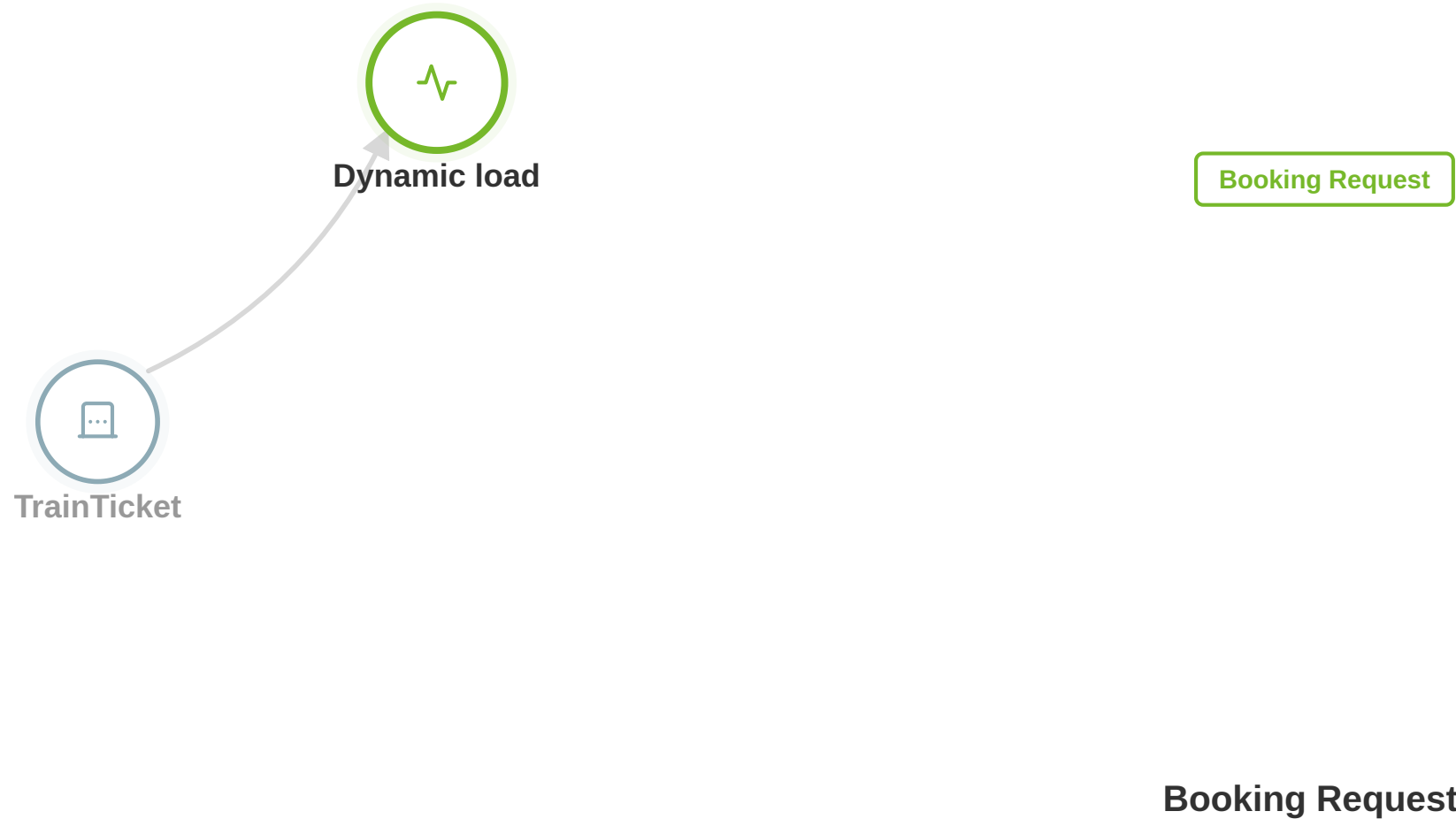
The benchmark generator is a closed loop

The benchmark generator is a closed loop

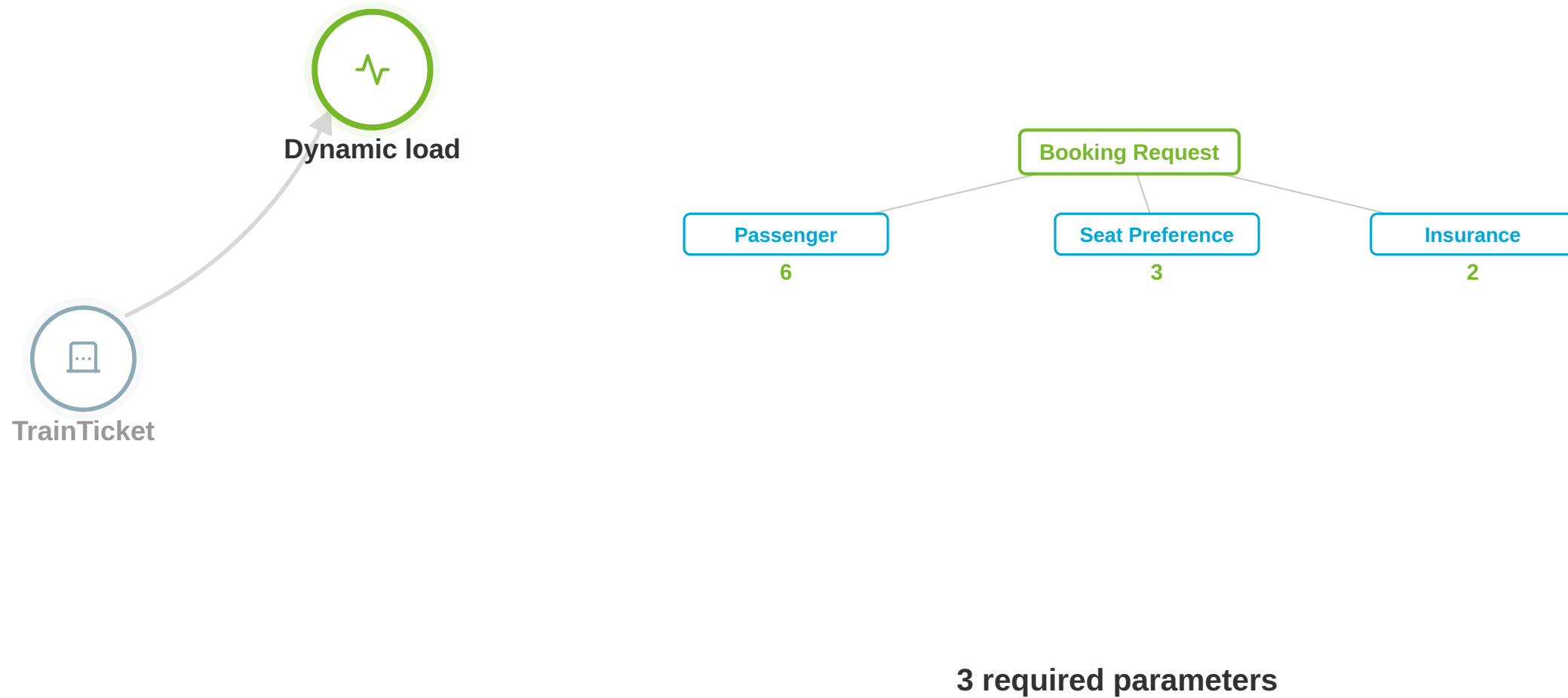


50 services with realistic dependencies

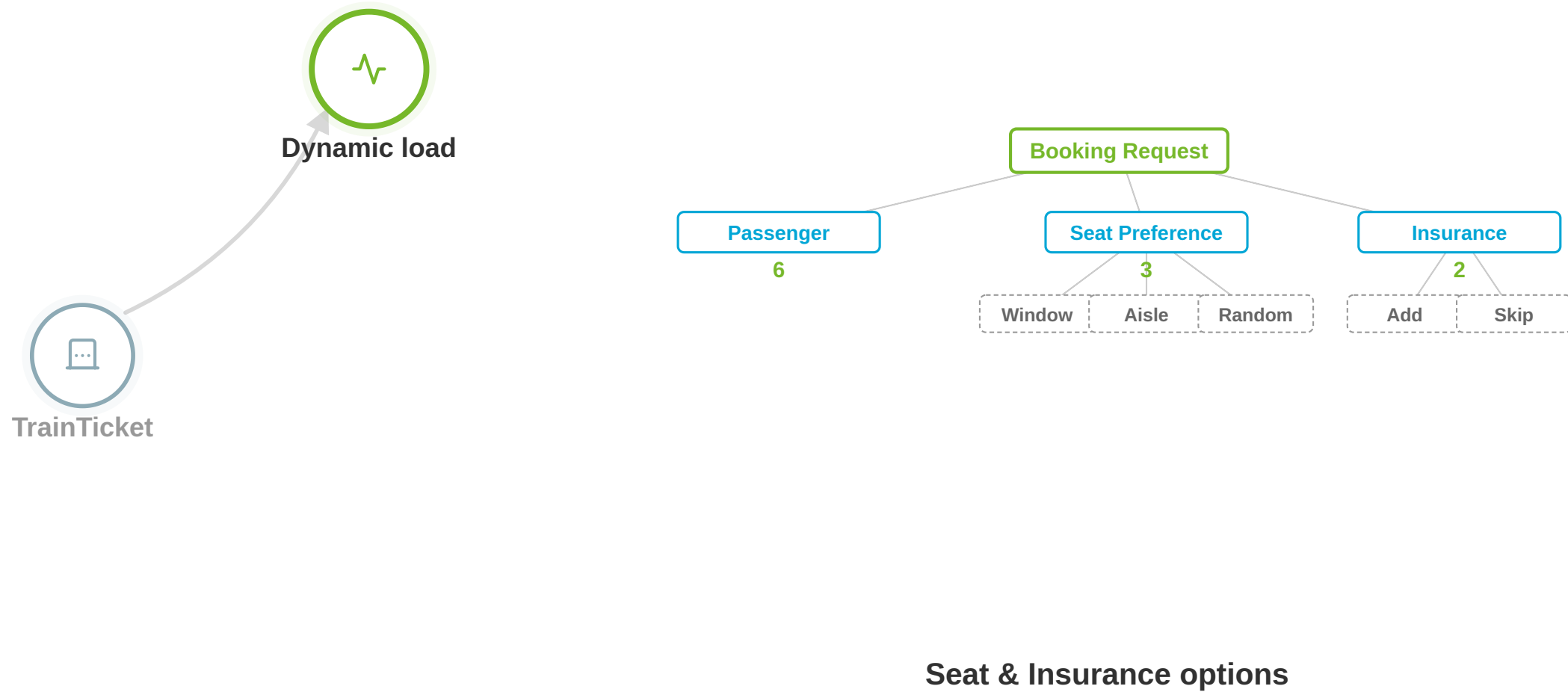
The benchmark generator is a closed loop



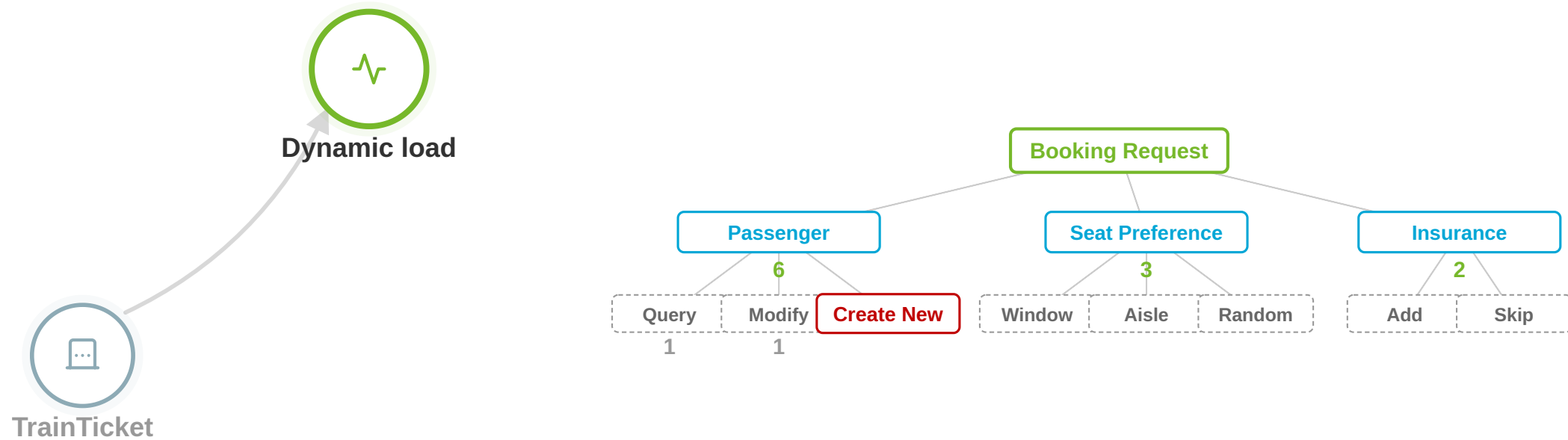
The benchmark generator is a closed loop



The benchmark generator is a closed loop

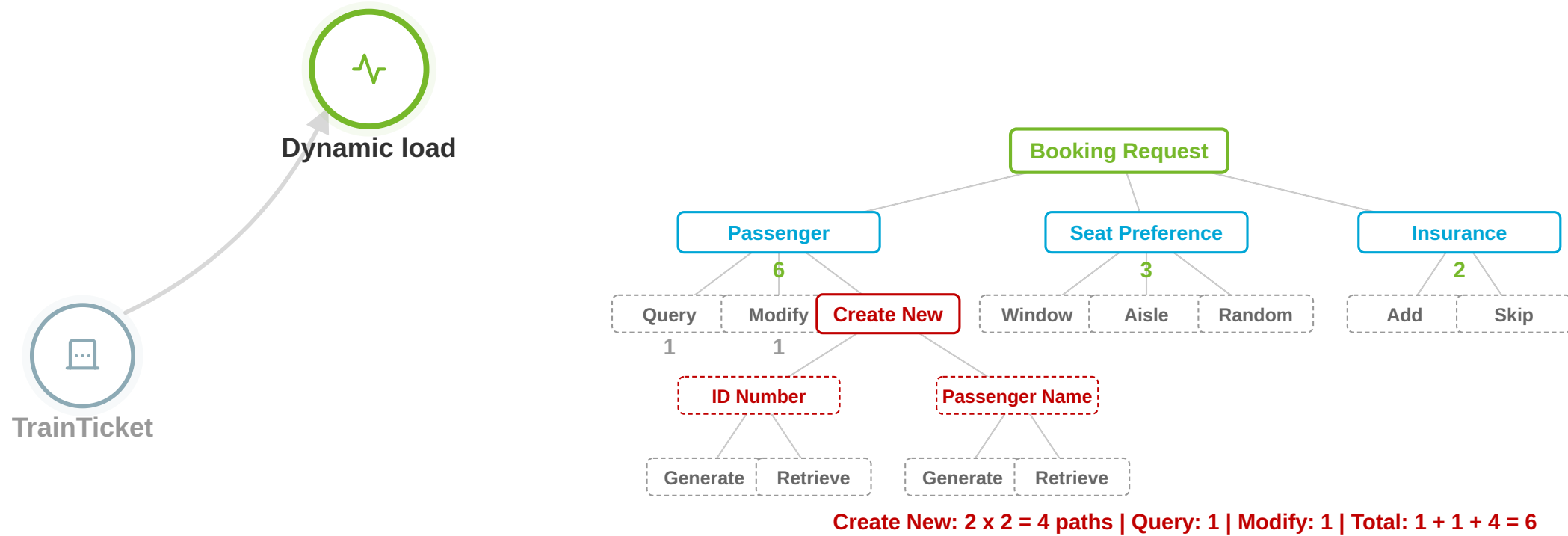


The benchmark generator is a closed loop



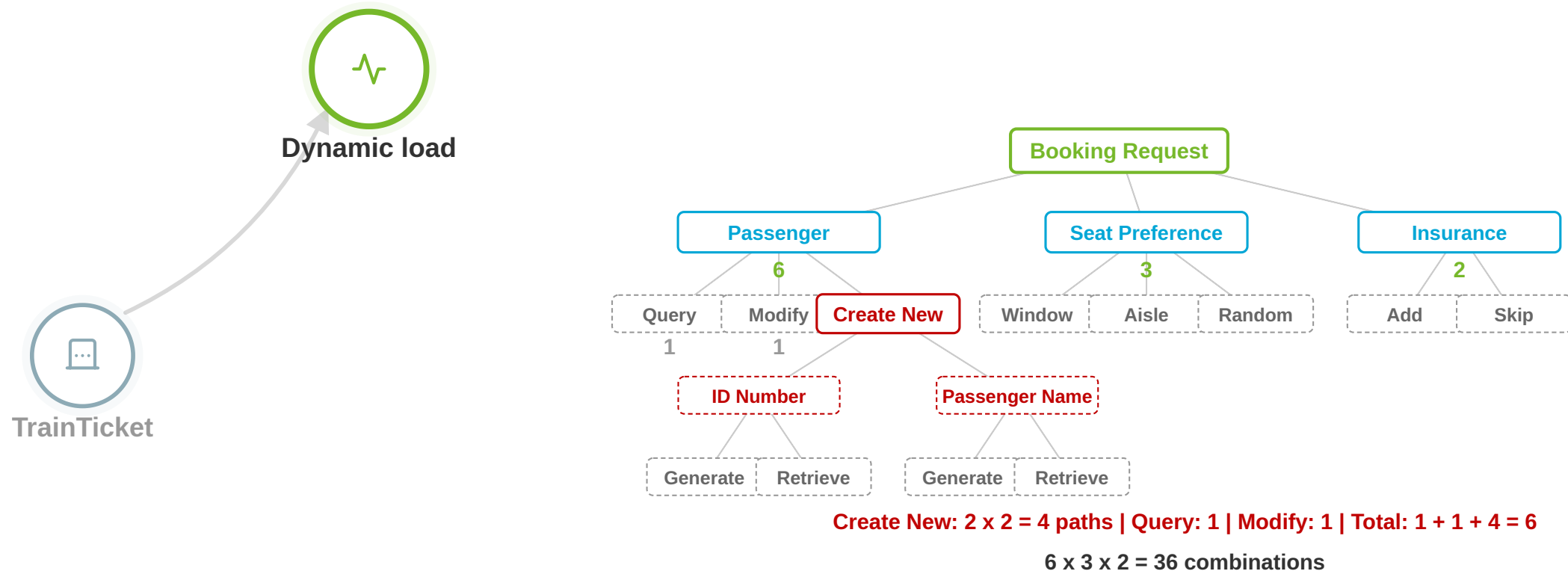
Passenger selection methods

The benchmark generator is a closed loop



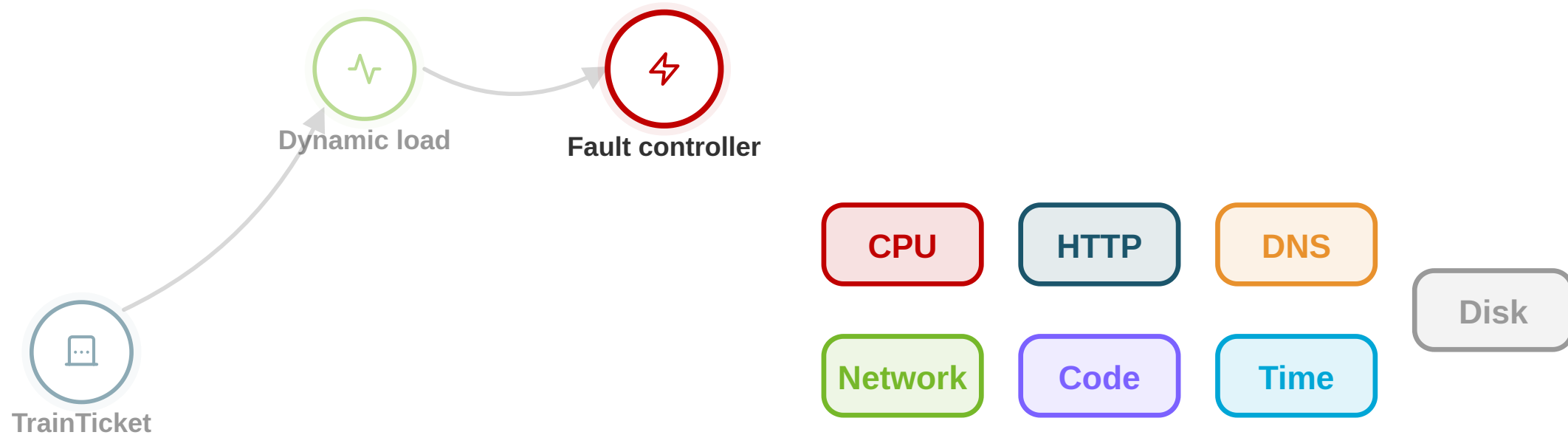
Recursive sub-workflows

The benchmark generator is a closed loop



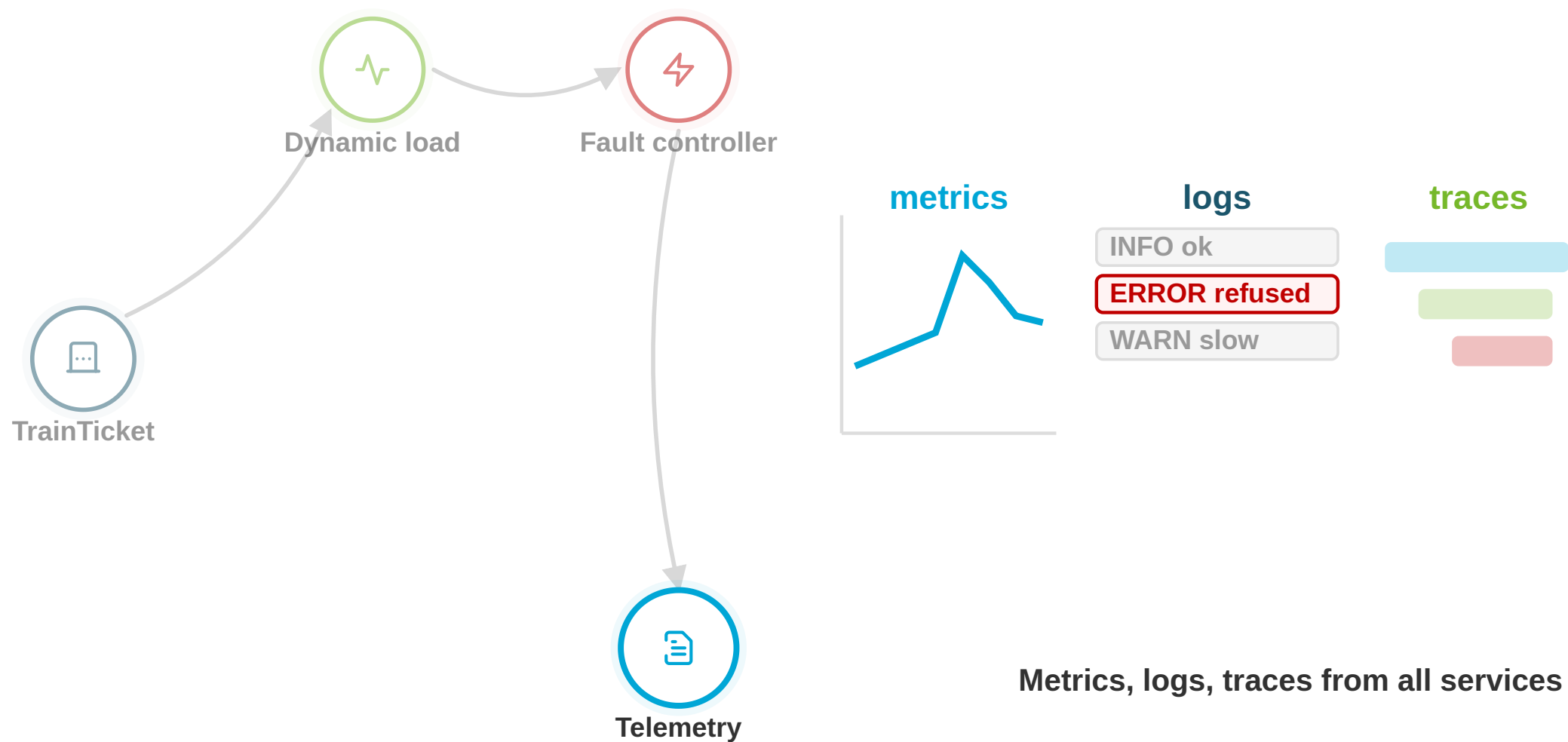
36 API combinations per booking flow

The benchmark generator is a closed loop

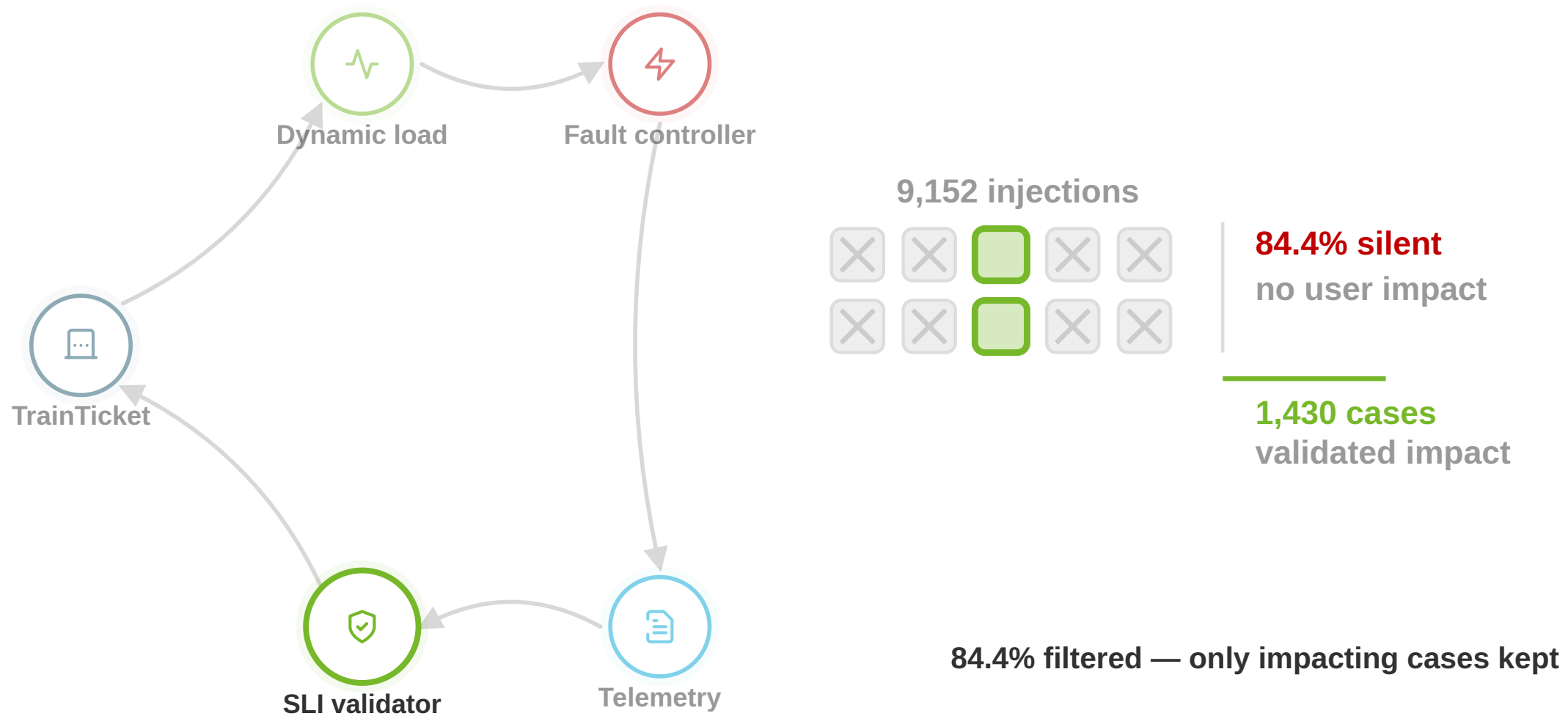


31 fault types across 7 categories

The benchmark generator is a closed loop



The benchmark generator is a closed loop



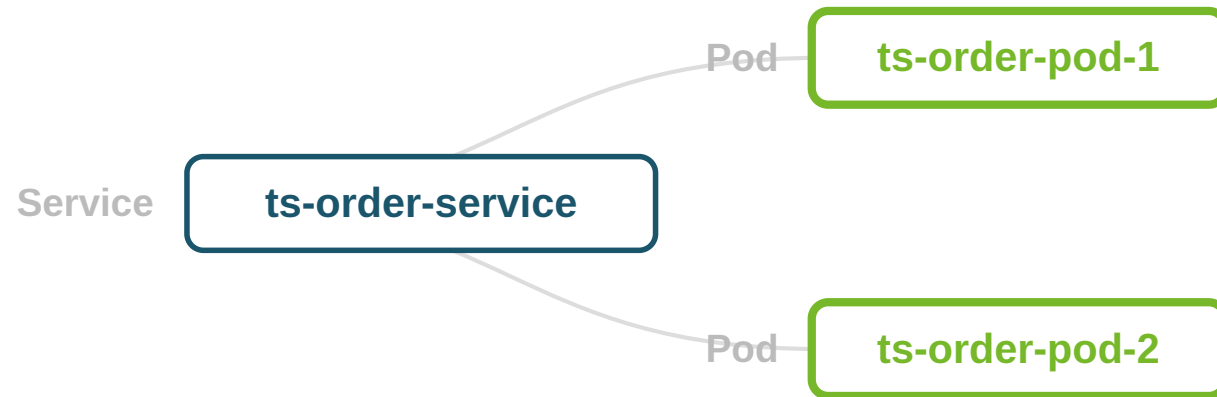
Ground truth is hierarchical

Ground truth is hierarchical

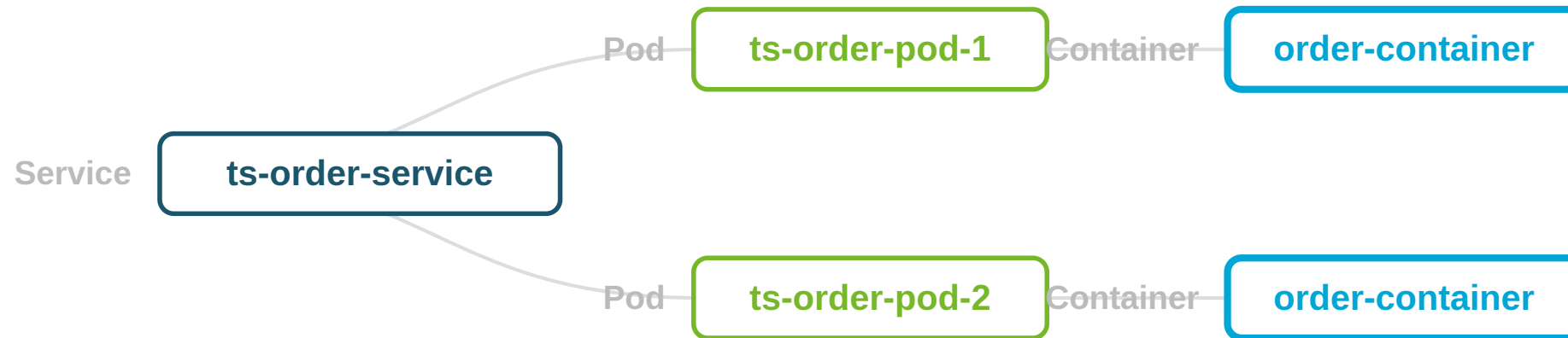
Service

ts-order-service

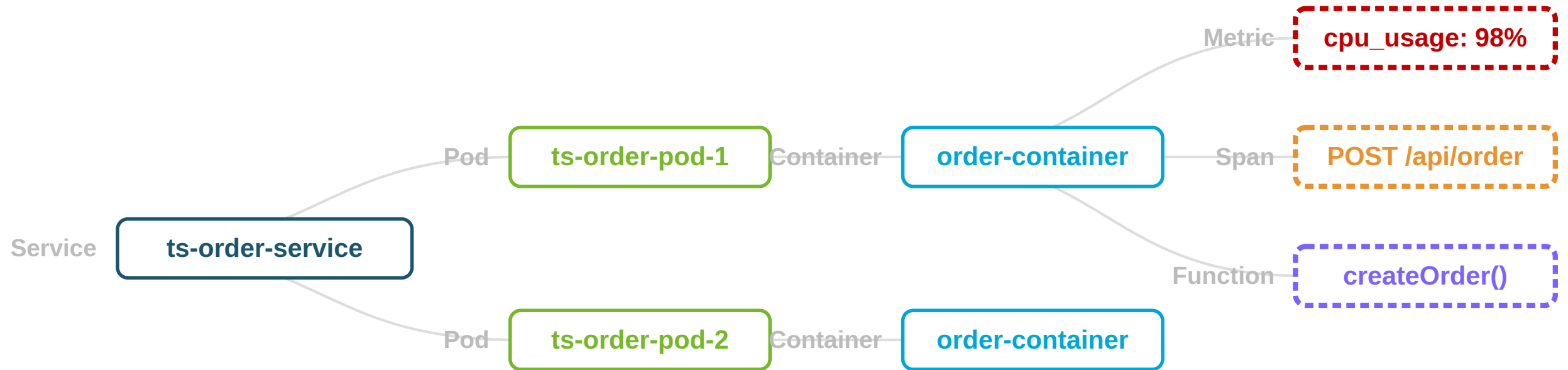
Ground truth is hierarchical



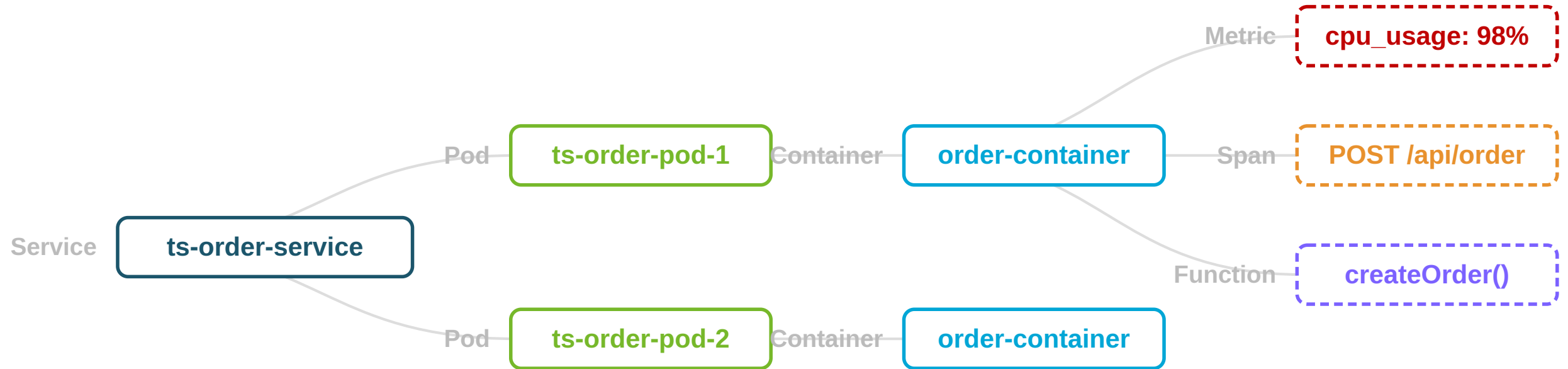
Ground truth is hierarchical



Ground truth is hierarchical



Ground truth is hierarchical



Service-level comparison for fair cross-method evaluation

Evaluation

Putting SOTA methods to the test



RQ4 Benchmark scale & diversity?

Evaluation

Putting SOTA methods to the test



RQ4 Benchmark scale & diversity?

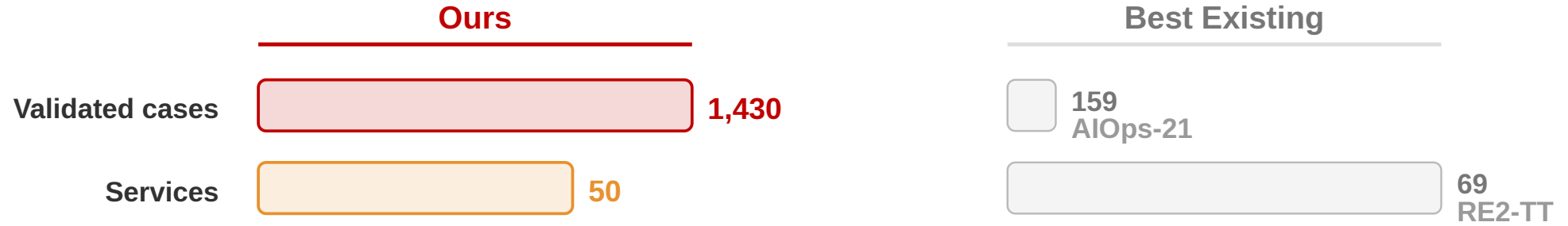
RQ5 SOTA performance & failure modes?

Our benchmark vs. existing datasets

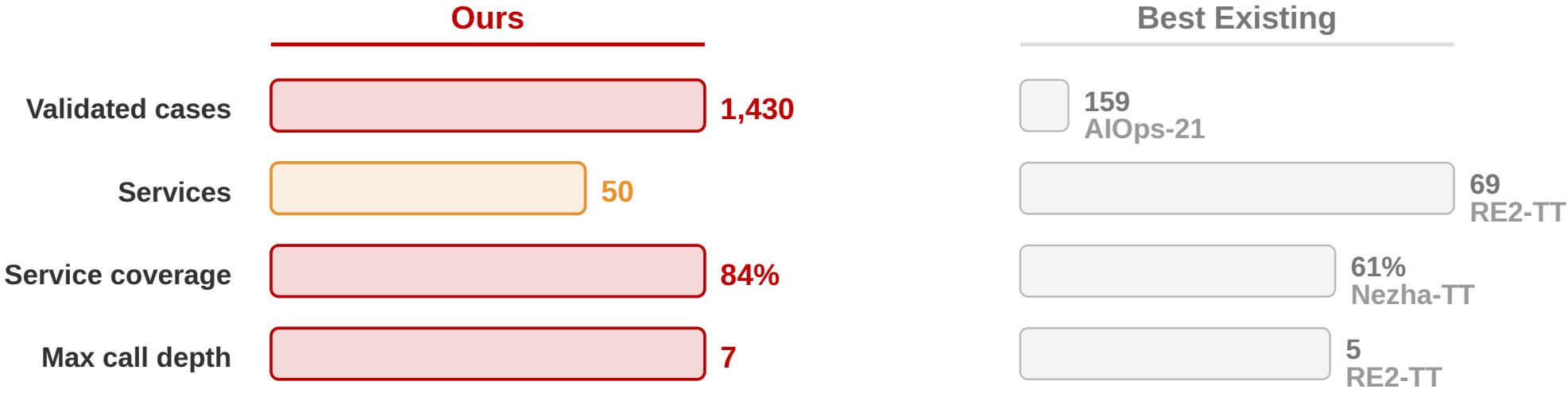
Ours

Best Existing

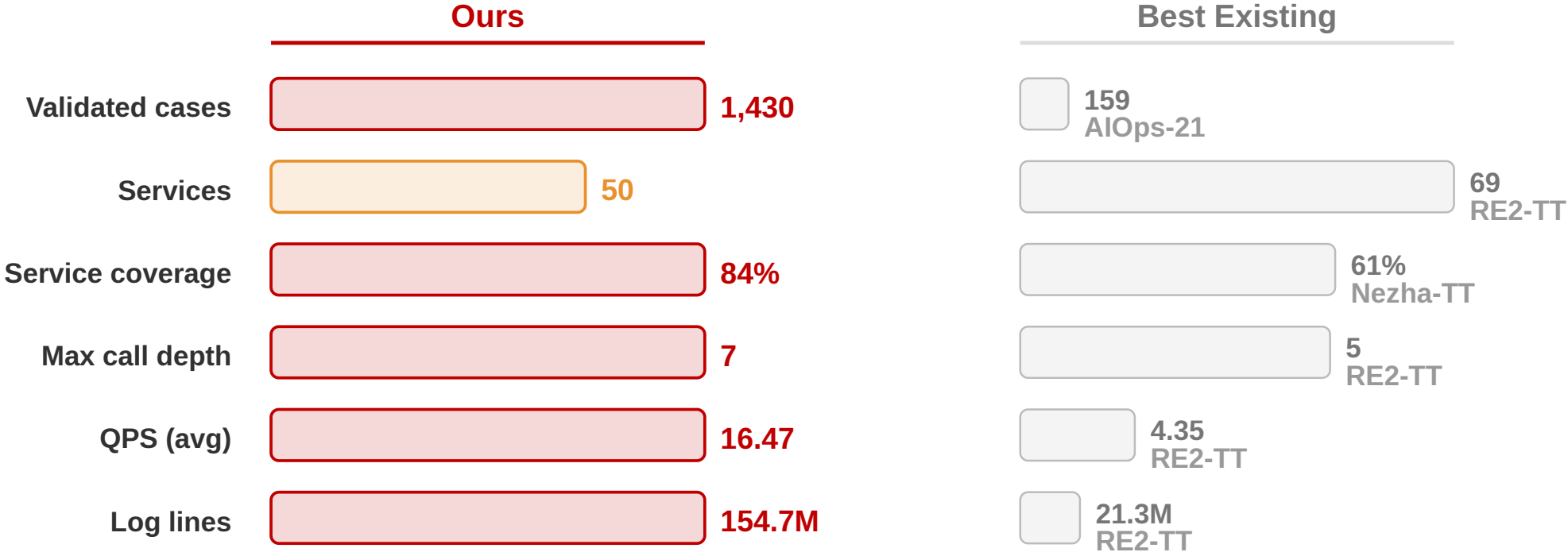
Our benchmark vs. existing datasets



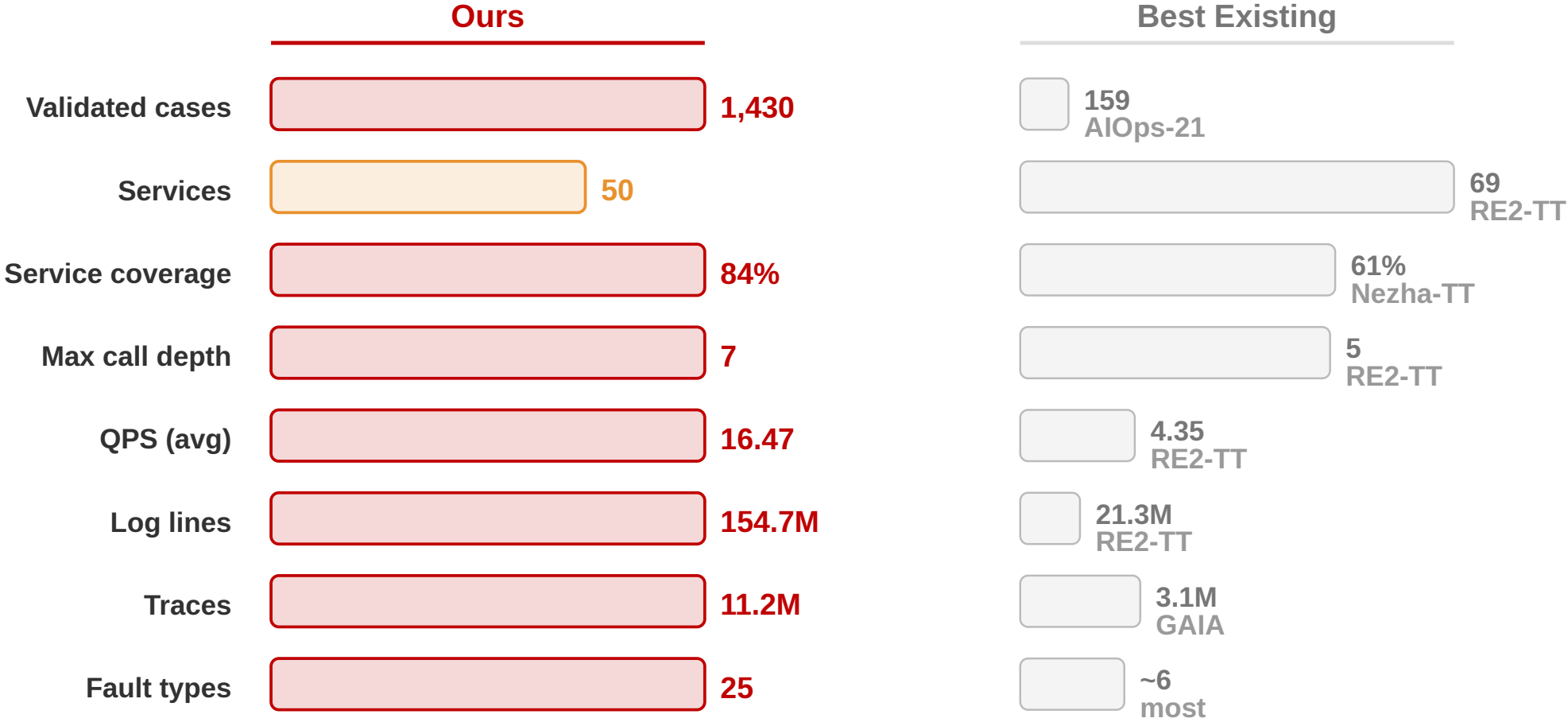
Our benchmark vs. existing datasets



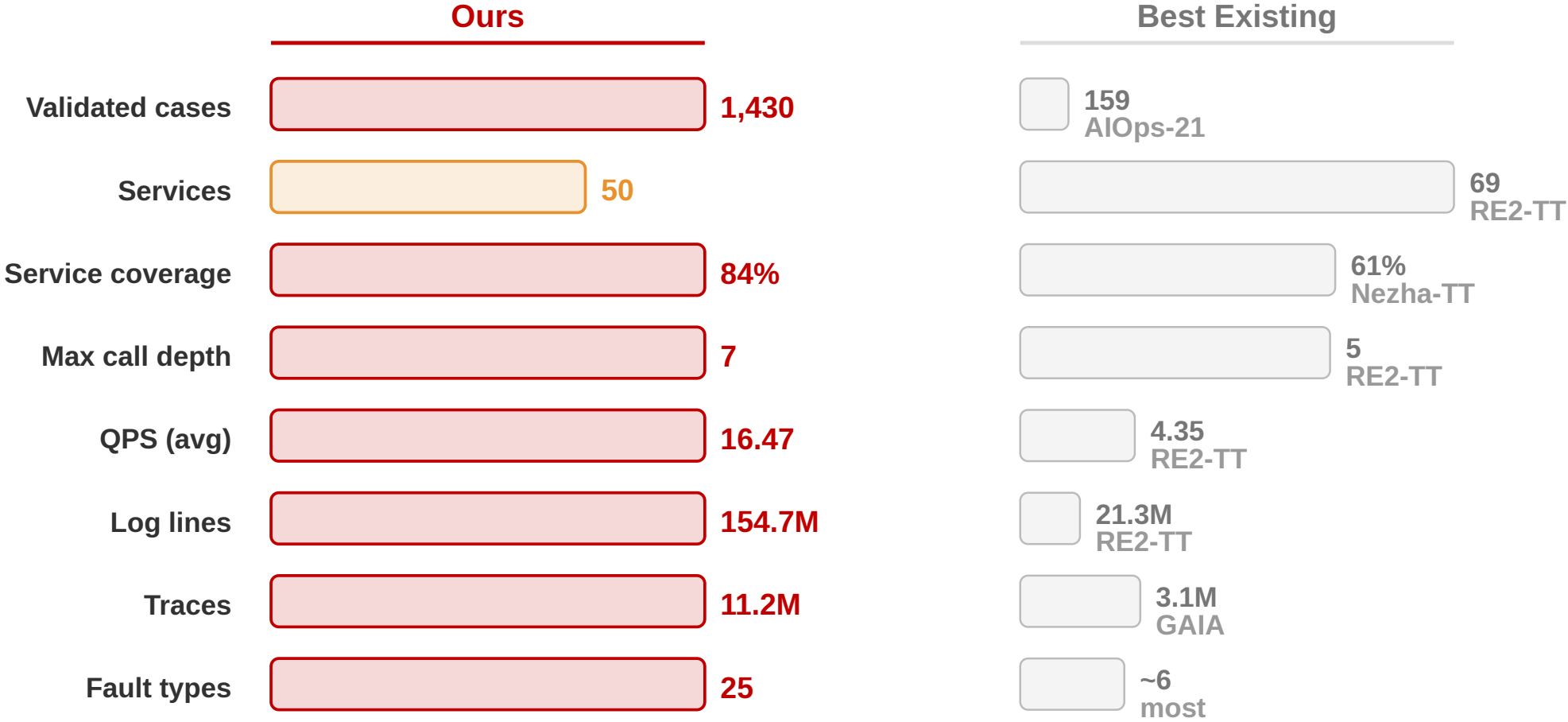
Our benchmark vs. existing datasets



Our benchmark vs. existing datasets



Our benchmark vs. existing datasets

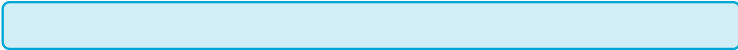
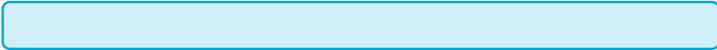


Realistic across all dimensions

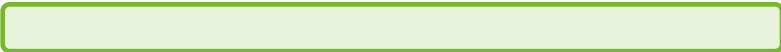
SOTA accuracy collapses on the new benchmark

Top@1 @1 @3 @5 Time(s)

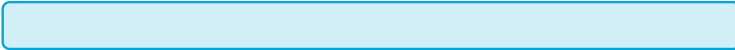
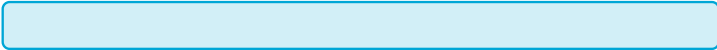
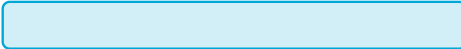
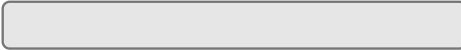
SOTA accuracy collapses on the new benchmark

	Top@1	@1	@3	@5	Time(s)
MicroRCA	 0.37	0.37	0.50	0.64	35.6
Baro	 0.36	0.36	0.50	0.58	1.0
MicroDig	 0.35	0.35	0.61	0.67	19.0
MicroHECL	 0.34	0.34	0.34	0.34	34.8

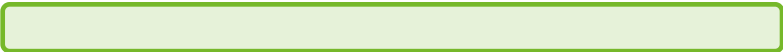
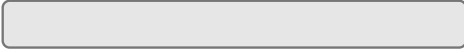
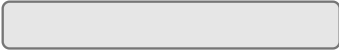
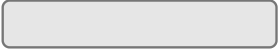
SOTA accuracy collapses on the new benchmark

	Top@1	@1	@3	@5	Time(s)
MicroRCA	 0.37	0.37	0.50	0.64	35.6
Baro	 0.36	0.36	0.50	0.58	1.0
MicroDig	 0.35	0.35	0.61	0.67	19.0
MicroHECL	 0.34	0.34	0.34	0.34	34.8
SimpleRCA	 0.28	0.28	0.60	0.80	0.9
CausalRCA	 0.22	0.22	0.40	0.42	927
Shapleyiq	 0.22	0.22	0.36	0.52	42.7
Eadro	 0.16	0.16	0.16	0.16	32.8

SOTA accuracy collapses on the new benchmark

	Top@1	@1	@3	@5	Time(s)
MicroRCA	 0.37	0.37	0.50	0.64	35.6
Baro	 0.36	0.36	0.50	0.58	1.0
MicroDig	 0.35	0.35	0.61	0.67	19.0
MicroHECL	 0.34	0.34	0.34	0.34	34.8
SimpleRCA	 0.28	0.28	0.60	0.80	0.9
CausalRCA	 0.22	0.22	0.40	0.42	927
Shapleyiq	 0.22	0.22	0.36	0.52	42.7
Eadro	 0.16	0.16	0.16	0.16	32.8
DiagFusion	 0.13	0.13	0.17	0.22	3.0
MicroRank	 0.04	0.04	0.18	0.25	36.3
Nezha	 0.04	0.04	0.08	0.08	94.6
Art	 0.04	0.04	0.08	0.10	8.1

SOTA accuracy collapses on the new benchmark

	Top@1	@1	@3	@5	Time(s)
MicroRCA	 0.37	0.37	0.50	0.64	35.6
Baro	 0.36	0.36	0.50	0.58	1.0
MicroDig	 0.35	0.35	0.61	0.67	19.0
MicroHECL	 0.34	0.34	0.34	0.34	34.8
SimpleRCA	 0.28	0.28	0.60	0.80	0.9
CausalRCA	 0.22	0.22	0.40	0.42	927
Shapleyiq	 0.22	0.22	0.36	0.52	42.7
Eadro	 0.16	0.16	0.16	0.16	32.8
DiagFusion	 0.13	0.13	0.17	0.22	3.0
MicroRank	 0.04	0.04	0.18	0.25	36.3
Nezha	 0.04	0.04	0.08	0.08	94.6
Art	 0.04	0.04	0.08	0.10	8.1

Best Top@1 = 0.37 · SimpleRCA = 0.28 · The shortcut no longer dominates

Model behavior changes across realistic fault types

Baro

SimpleRCA

MicroRCA

MicroDig

Shapleyiq

MicroHECL

CausalRCA

MicroRank

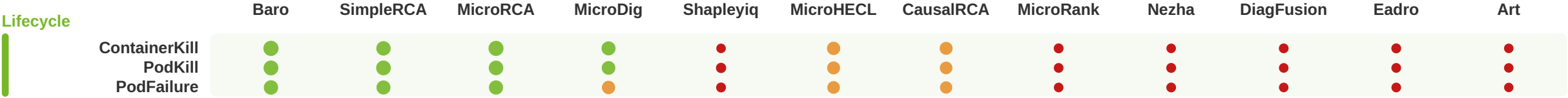
Nezha

DiagFusion

Eadro

Art

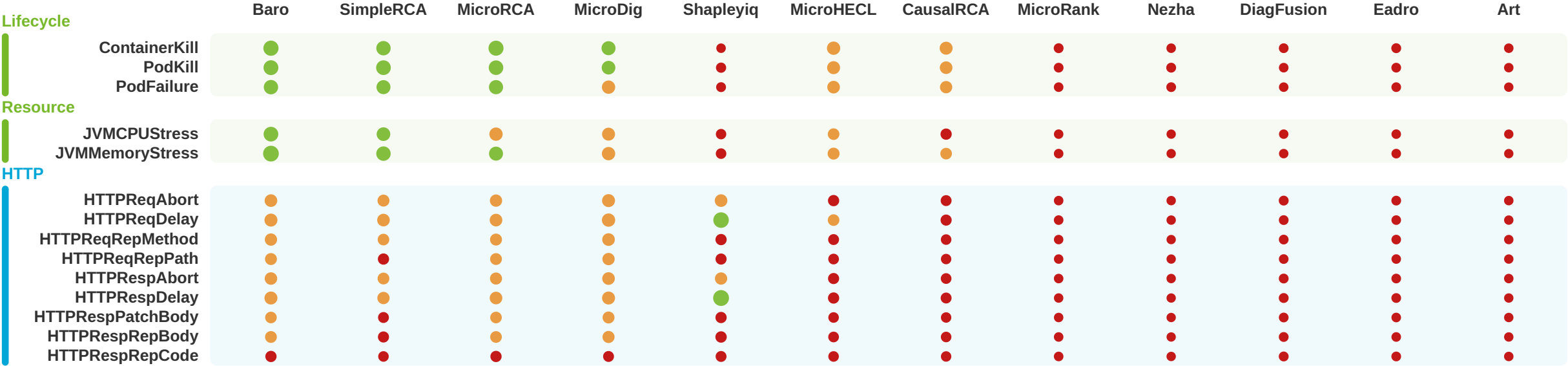
Model behavior changes across realistic fault types



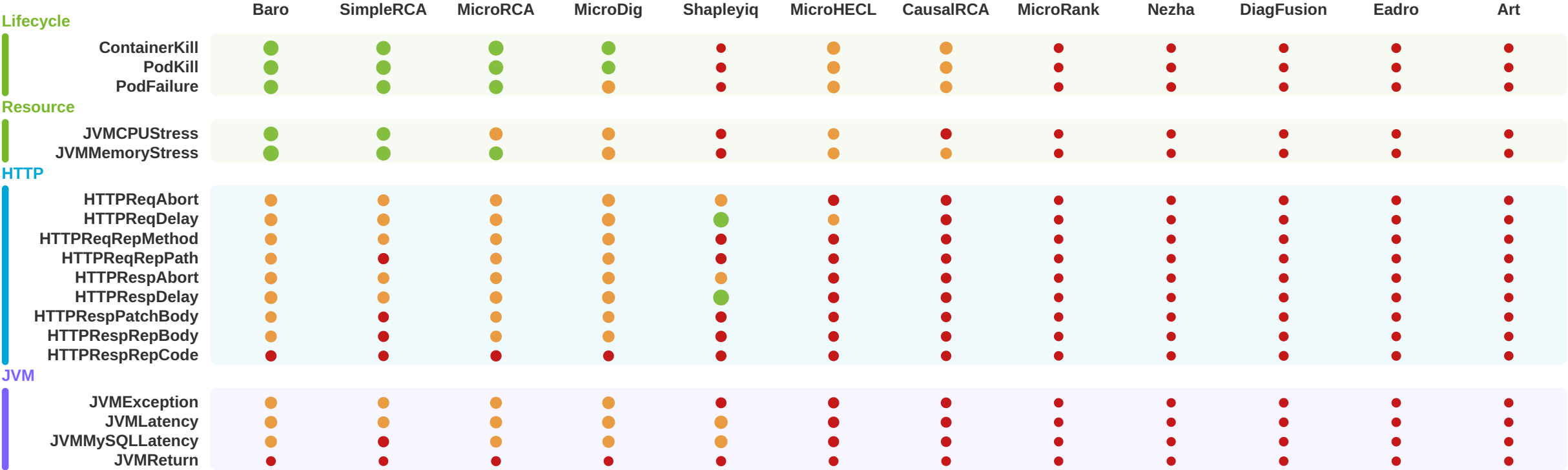
Model behavior changes across realistic fault types

		Baro	SimpleRCA	MicroRCA	MicroDig	Shapleyiq	MicroHECL	CausalRCA	MicroRank	Nezha	DiagFusion	Eadro	Art
Lifecycle	ContainerKill	●	●	●	●	●	●	●	●	●	●	●	●
	PodKill	●	●	●	●	●	●	●	●	●	●	●	●
	PodFailure	●	●	●	●	●	●	●	●	●	●	●	●
Resource	JVMCPUSstress	●	●	●	●	●	●	●	●	●	●	●	●
	JVMMemoryStress	●	●	●	●	●	●	●	●	●	●	●	●

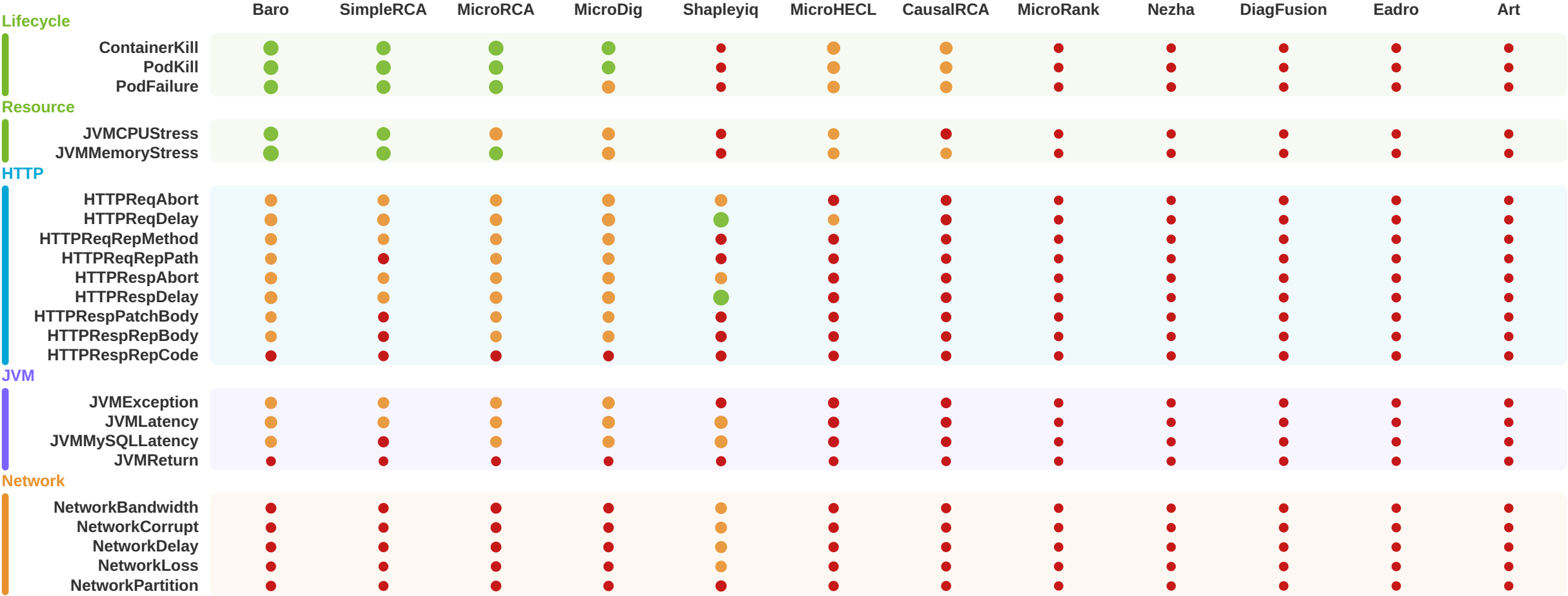
Model behavior changes across realistic fault types



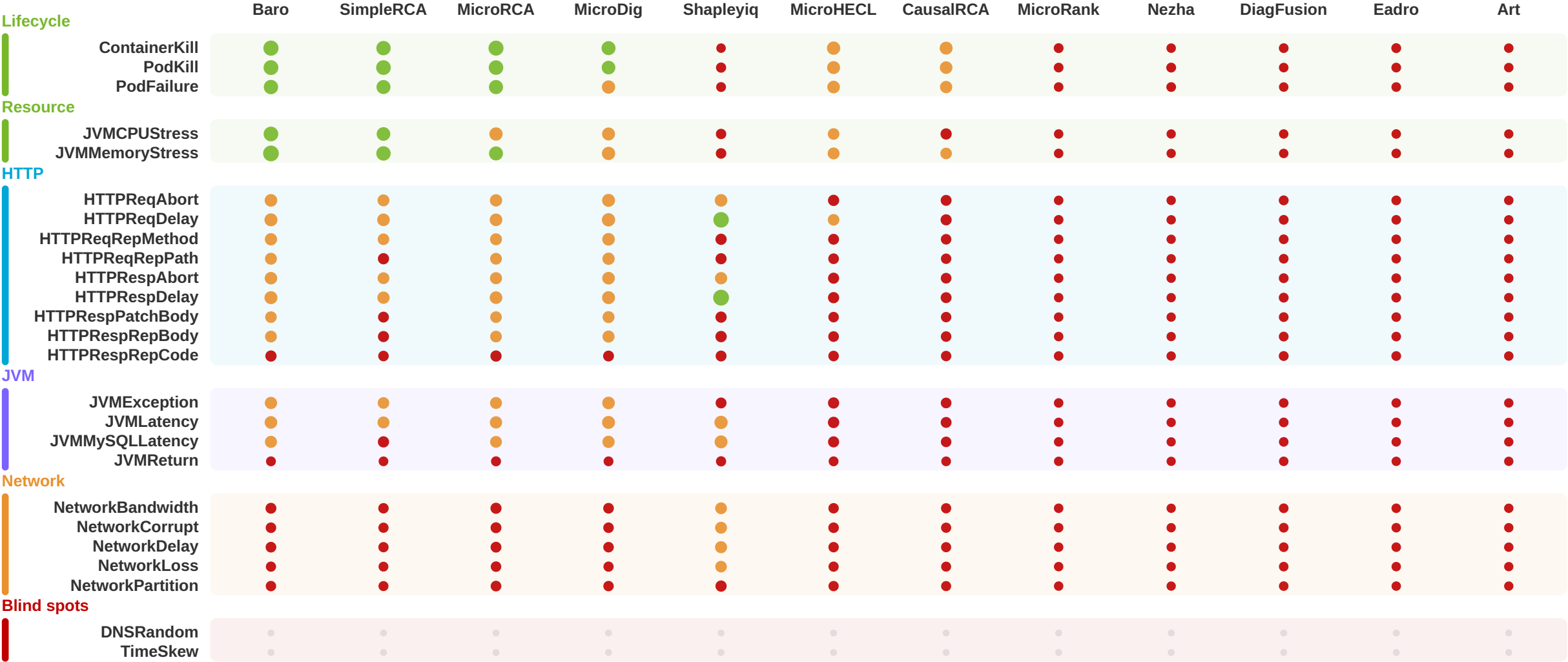
Model behavior changes across realistic fault types



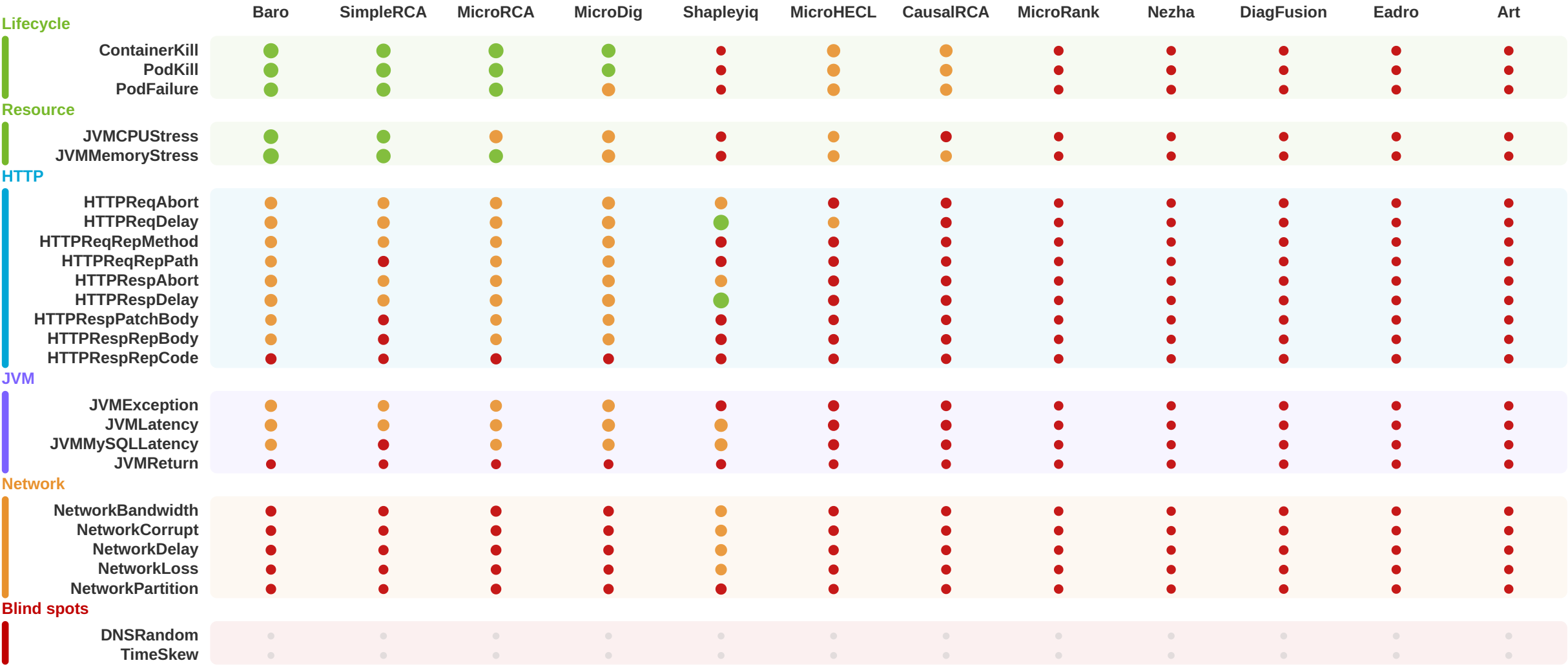
Model behavior changes across realistic fault types



Model behavior changes across realistic fault types



Model behavior changes across realistic fault types



No universal winner — performance depends on fault type

The failures cluster into three bottlenecks

The failures cluster into three bottlenecks

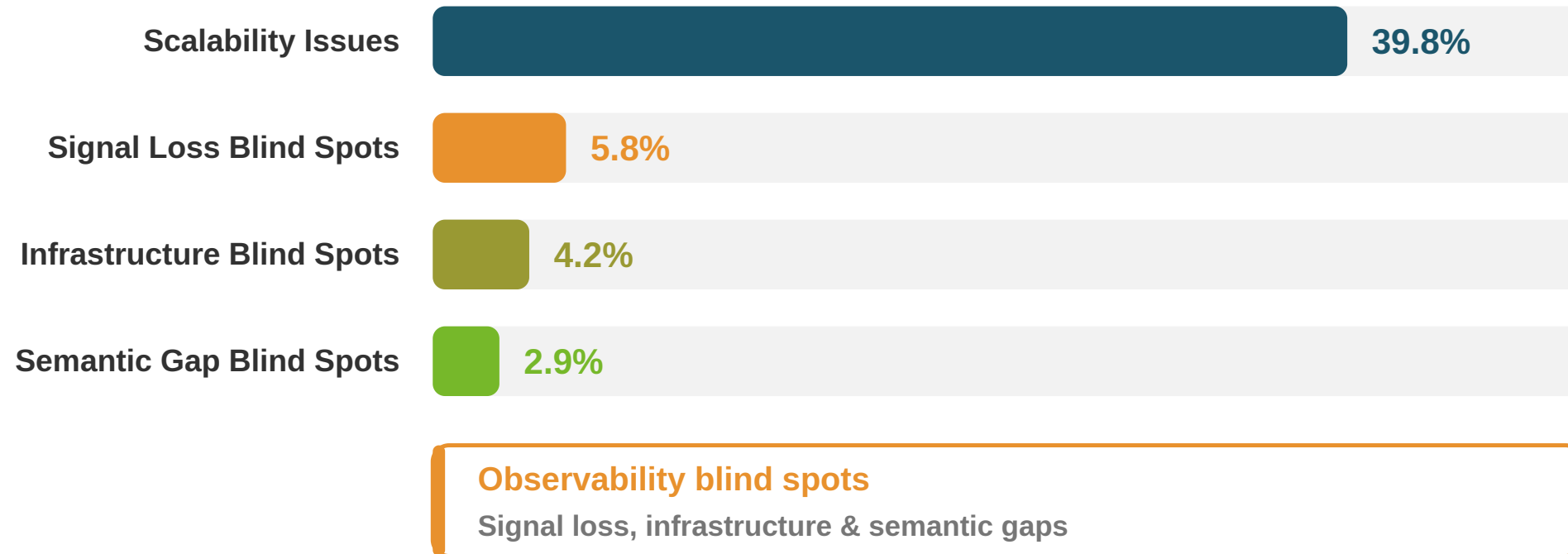
Scalability Issues

39.8%

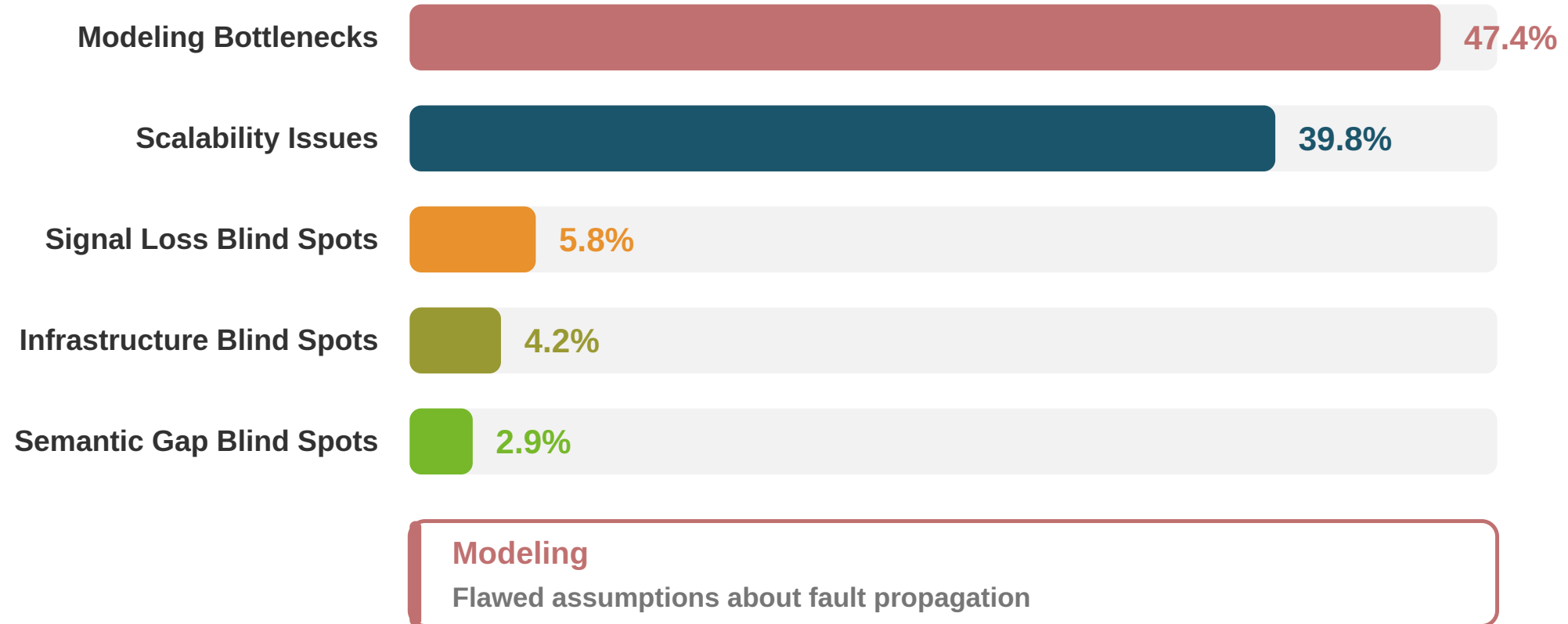
Scalability

CausalRCA: 927s, Nezha: 94.6s per case

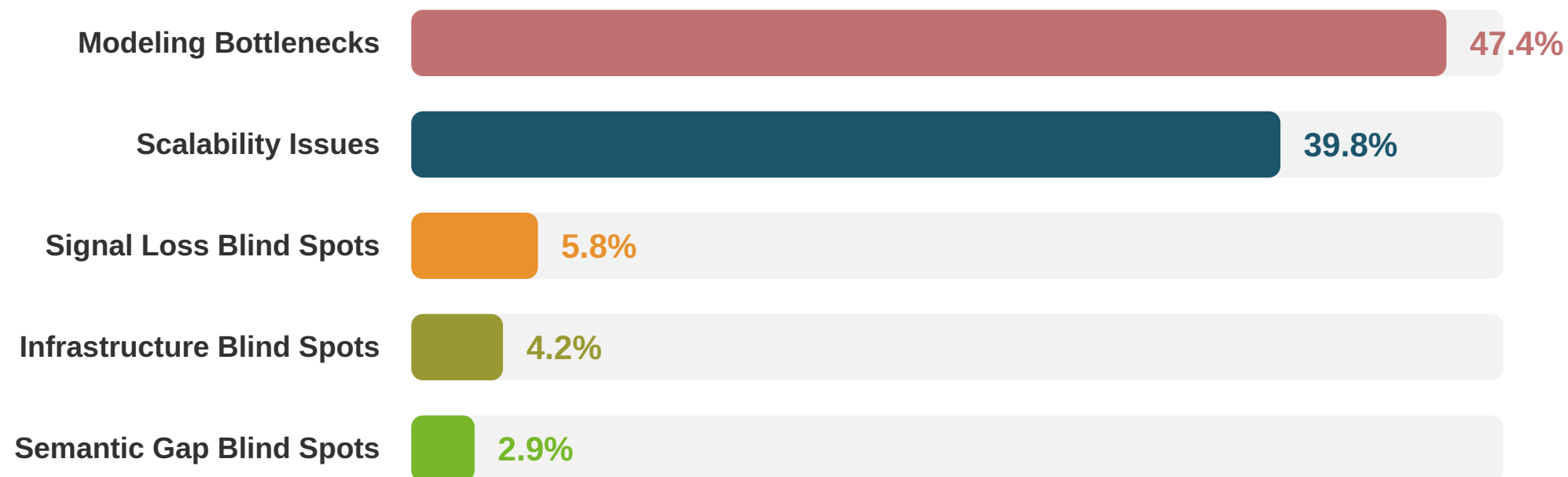
The failures cluster into three bottlenecks



The failures cluster into three bottlenecks



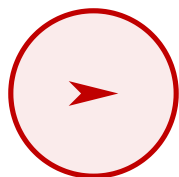
The failures cluster into three bottlenecks



262 hardest cases analyzed — three orthogonal bottlenecks

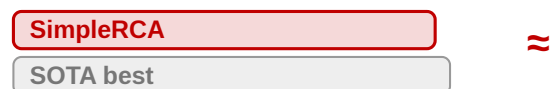
Takeaways

Takeaways



Benchmark shapes research

Old benchmarks

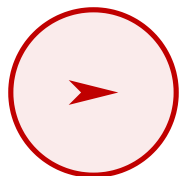


New benchmark



7/11 datasets shortcutted

Takeaways



Benchmark shapes research

Old benchmarks



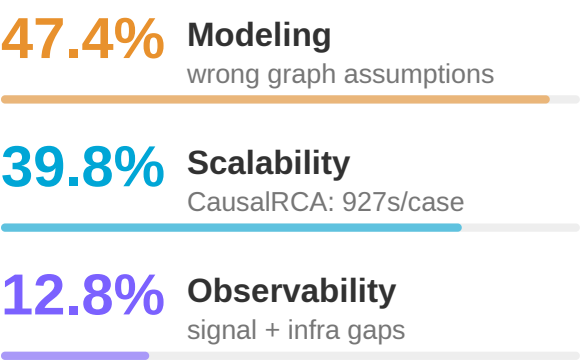
New benchmark



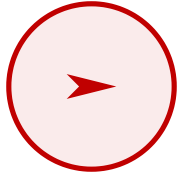
7/11 datasets shortcutted



Three orthogonal bottlenecks



Takeaways



Benchmark shapes research

Old benchmarks



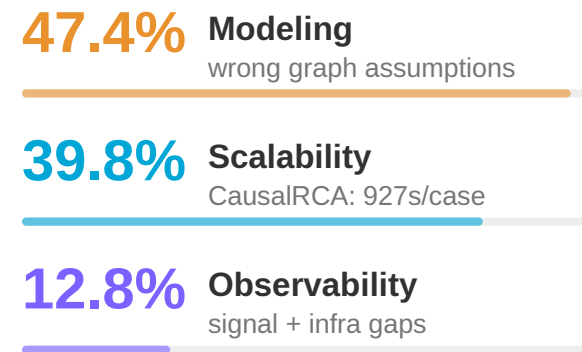
New benchmark



7/11 datasets shortcutted



Three orthogonal bottlenecks



Open benchmark & tools

1,430 validated cases

25 fault types

12 algorithms evaluated

Takeaways



operationspai.github.io/revisiting-rca-evaluation

Scan to access benchmark & code